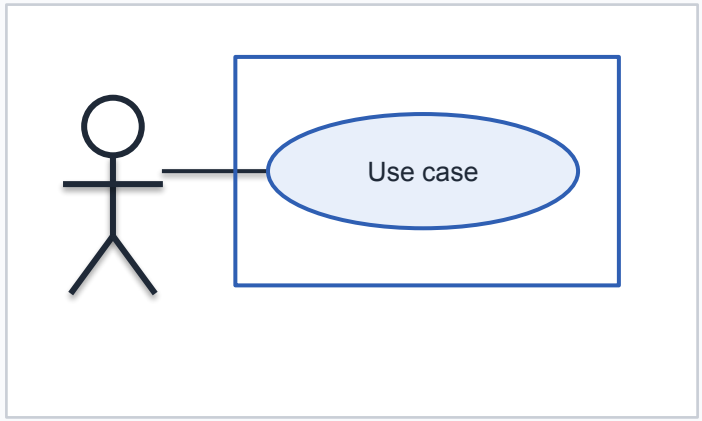
BRAC UNIVERSITY
Inspiring Excellence

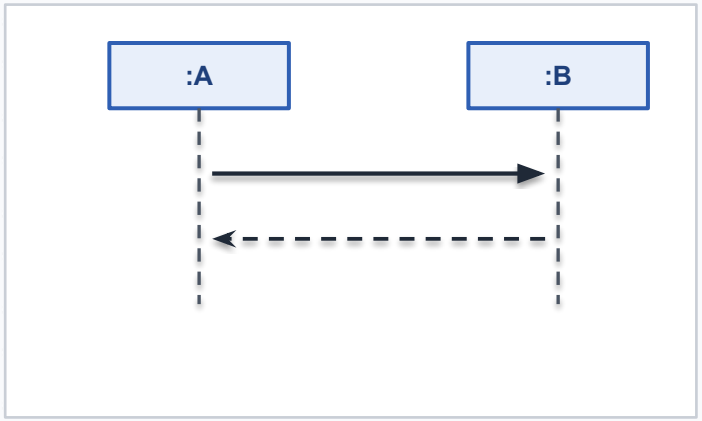
CSE471 · SYSTEM ANALYSIS AND DESIGN · LECTURE 04

Use Cases, Sequence and Activity Diagrams

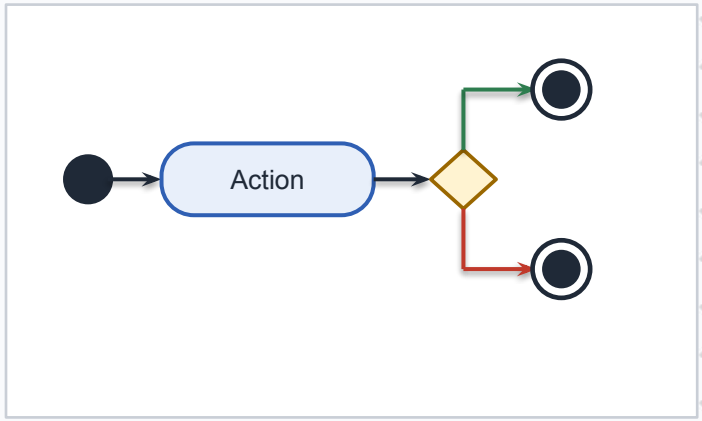
UML modelling, built on one running example: course registration



Who uses it?



Who talks to whom?



What are the steps?

Ref: Whitten et al., Systems Analysis and Design Methods, 7e. McGraw-Hill Higher Education

BRAC UNIVERSITY
Inspiring Excellence

Today's path

1. The problem
2. What a use case is
3. Use case diagram
4. Read the diagram
5. Sequence diagram
6. Activity diagram
7. Compare the three
8. Practice and recap

One example runs through every step: a student registering for a course section.

The problem we will model

A story

Nadia, a second-year student, wants to take **CSE471, section 10.**

She selects the section in the course registration system.

The system checks that she has passed **CSE470** (the prerequisite) and that a **seat is free.**

Then it either **confirms her enrollment** or **explains why it failed.**

Requirements we will use

R1 Students log in.

R2 Students register for a section.

R3 The system checks prerequisites with the Student Records system.

R4 If the section is full, the student may join a waitlist.

R5 Students drop sections, view their schedule and pay fees through a payment gateway.

R6 Registrar staff manage sections and seat limits.

What is UML?

Unified Modeling Language

- A visual language with a standard notation that everyone on a software project can understand: clients, analysts, designers and developers.
- Started at Rational Software in 1994–95.
- **Popular tools:** Rational Rose, Microsoft Visio, draw.io

Why a shared language?

- A drawing is quicker to check than pages of text.
- Everyone reads the same symbol the same way.
- Mistakes in the requirements show up before any code is written.

BRAC UNIVERSITY
Inspiring Excellence

The UML diagram family

UML 2 groups its diagrams into two families. Today's three are highlighted.

Structure diagrams: what the system is made of	Behaviour diagrams: what the system does
Class · later in the course	**Use case · today**
Object	**Activity · today**
Component	State machine
Deployment	Interaction diagrams
Package	**Sequence · today**
Composite structure	Communication
Profile	Interaction overview
	Timing

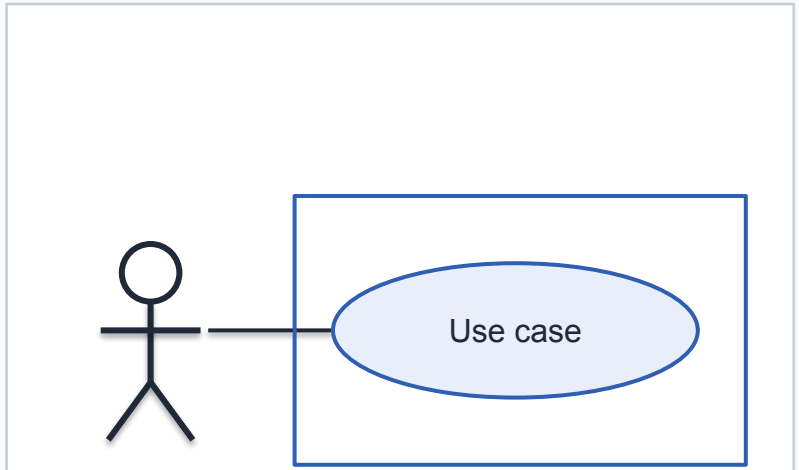
BRAC UNIVERSITY
Inspiring Excellence

01

PART 1

Use cases and use case diagrams

Who uses the system, and for what?



use case diagram

What is a use case?



One use case = **one goal** an **actor** reaches by using the **system.**

- **A requirements-analysis tool:** it shows the uses of the system, case by case.
- **"Use"** is the action. **"Case"** links that action to the actor who performs it.
- **Action verbs in the requirements are candidate use cases:** log in, register, drop, pay, manage.
- **To draw one, find:** (1) everyone who performs actions, and (2) every action the system must support.

Find the actor, goal and boundary

PRACTICE 1

Use the story and requirements from step 1.

a. Who is the primary actor?

b. What is that actor's goal? Write it as a use case name.

c. Draw the system boundary: what is inside the system, and what is outside?

d. Name one other system our system must talk to.

Think for 1 minute, then discuss with your neighbour.

Actor, goal and boundary

ANSWER 1

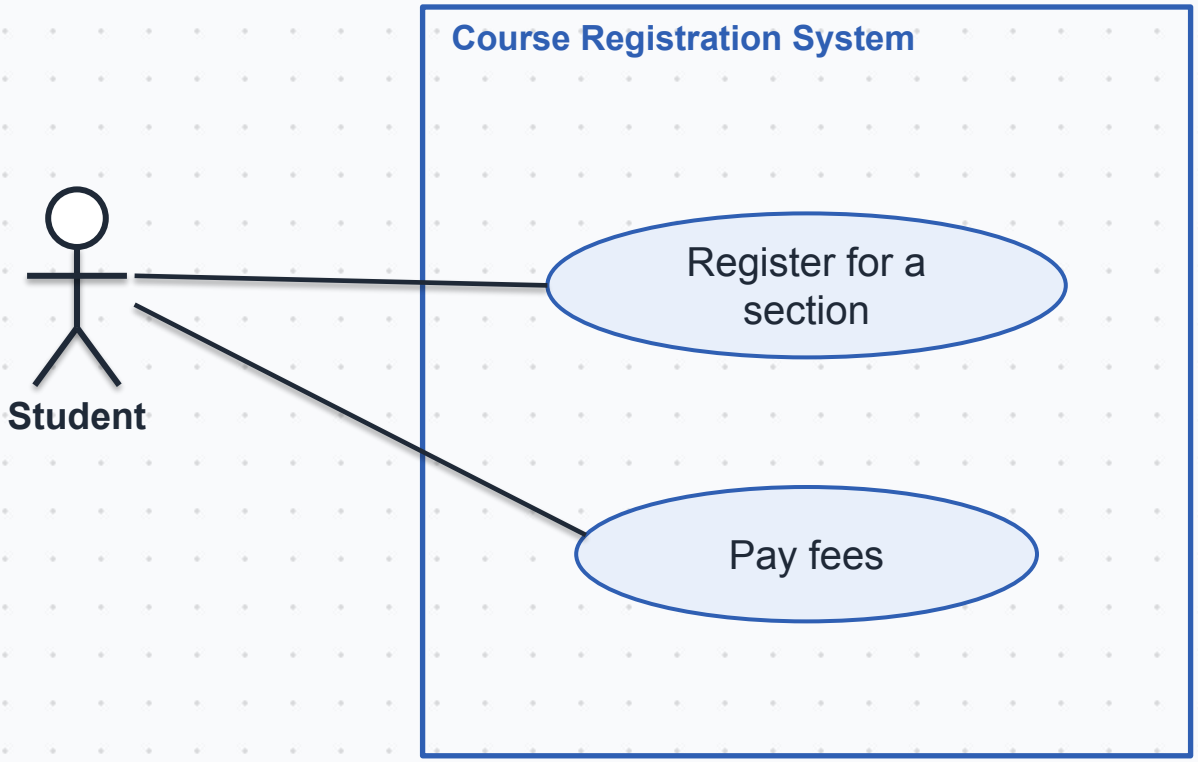
a **Student**
Why: starts the interaction and gets the value from it.

b **Register for a section**
Why: verb first, names a complete goal, not a button click.

c **Inside: registration rules, seat counts, enrollment records. Outside: Student, Registrar staff, other systems.**
Why: anything we are not building sits outside.

d **Student Records system**
Why: it holds which courses Nadia has passed. It is an actor of type “other system”.

Four parts of a use case diagram



1 · Actor
Someone or something outside that uses the system.

2 · Use case
A goal the system supports. Drawn as an ellipse.

3 · System boundary
A rectangle named after the system. Use cases inside, actors outside.

4 · Relation
A line connecting two parts. Four kinds, next slides.

Actors

- Performs actions or roles in the system.
- Gives input to the system.
- Uses the processed output to do their work.
- **Must be connected to at least one use case.**
- Always drawn outside the system boundary.

Primary actor

Performs the main system functions.

e.g. Student registering for a section (in a ride app: the rider).

Secondary actor

Performs administrative functions.

e.g. Registrar staff setting seat limits (in a ride app: a manager setting discounts).

External hardware

A hardware device that is part of the application.

e.g. ID-card scanner at the registrar's desk.

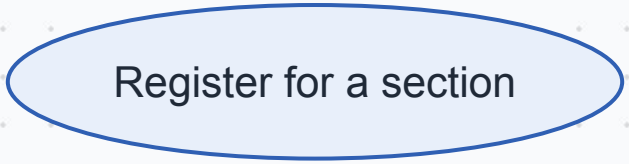
Other system

An external system the system talks to.

e.g. Payment gateway; Student Records system.

Use cases

- Shows a system function performed for an actor.
- Can also describe a sequence of actions in the system.
- Every use case has a unique name.
- **The name starts with a verb.**
- Sits inside the system boundary.
- Is connected to an actor or to another use case.
- Is drawn as an ellipse.



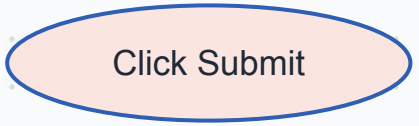
Good: verb + object, a whole goal



a noun, not an action



describes system internals, not a user goal



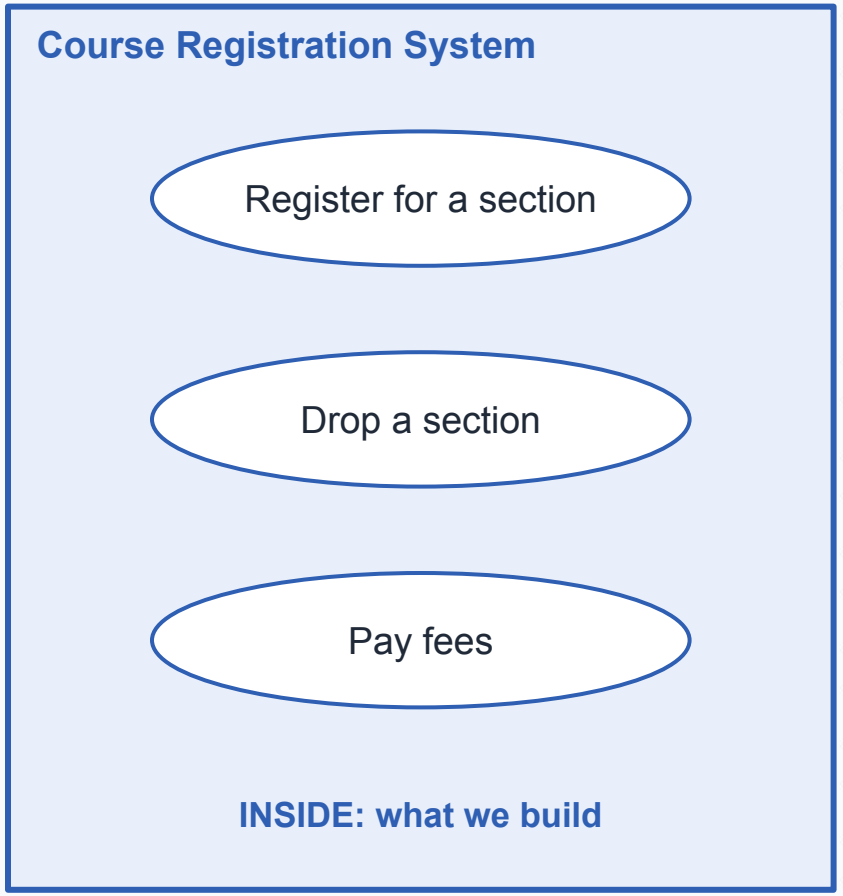
one tiny screen step, not a goal

System boundary



Student

OUTSIDE: people



A rectangle named after the system.
Shows where the system meets its users.

«system»
Payment Gateway

OUTSIDE: other systems

Relations: four kinds of line

Include and extend are the two kinds of dependency between use cases.

Association actor — use case	solid line, no arrowhead	
Generalization parent — child (actors or use cases)	solid line, hollow triangle at the parent	
Include base use case always uses another	dashed arrow «include», pointing to the included use case	
Extend optional behaviour added to a base	dashed arrow «extend», pointing to the base use case	

Association and generalization

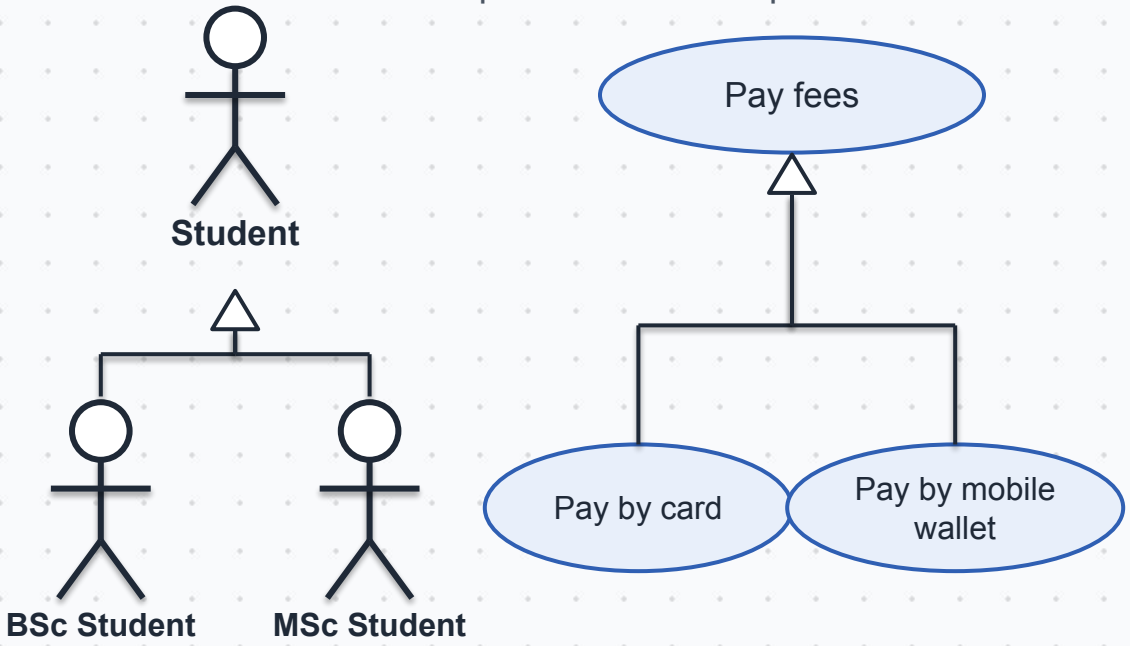
Association



- Connects an actor with a use case.
- Straight solid line. No arrow, not dashed, not curved.
- Every actor has at least one association.
- A use case can have zero or more actors.

Generalization

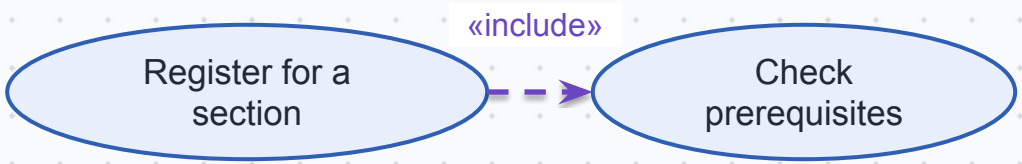
Parent–child: the child is a special kind of the parent.



Solid line with a hollow triangle at the parent. Works for actors (left) and for use cases (right).

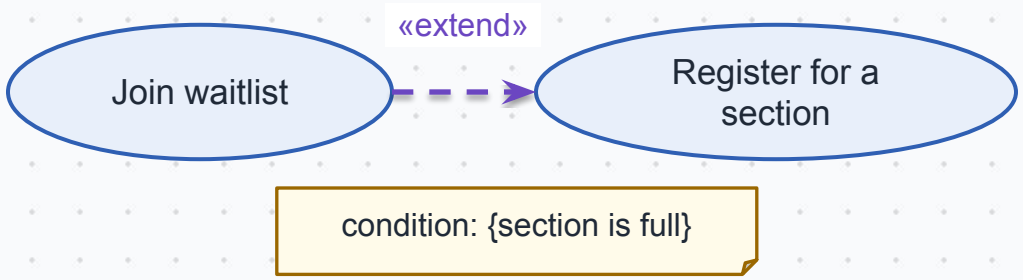
Include and extend

Include: always happens



- The base use case always runs the included one.
- Supports reuse: “Swap a section” can include it too.
- **Arrow points to the included use case.**

Extend: sometimes happens



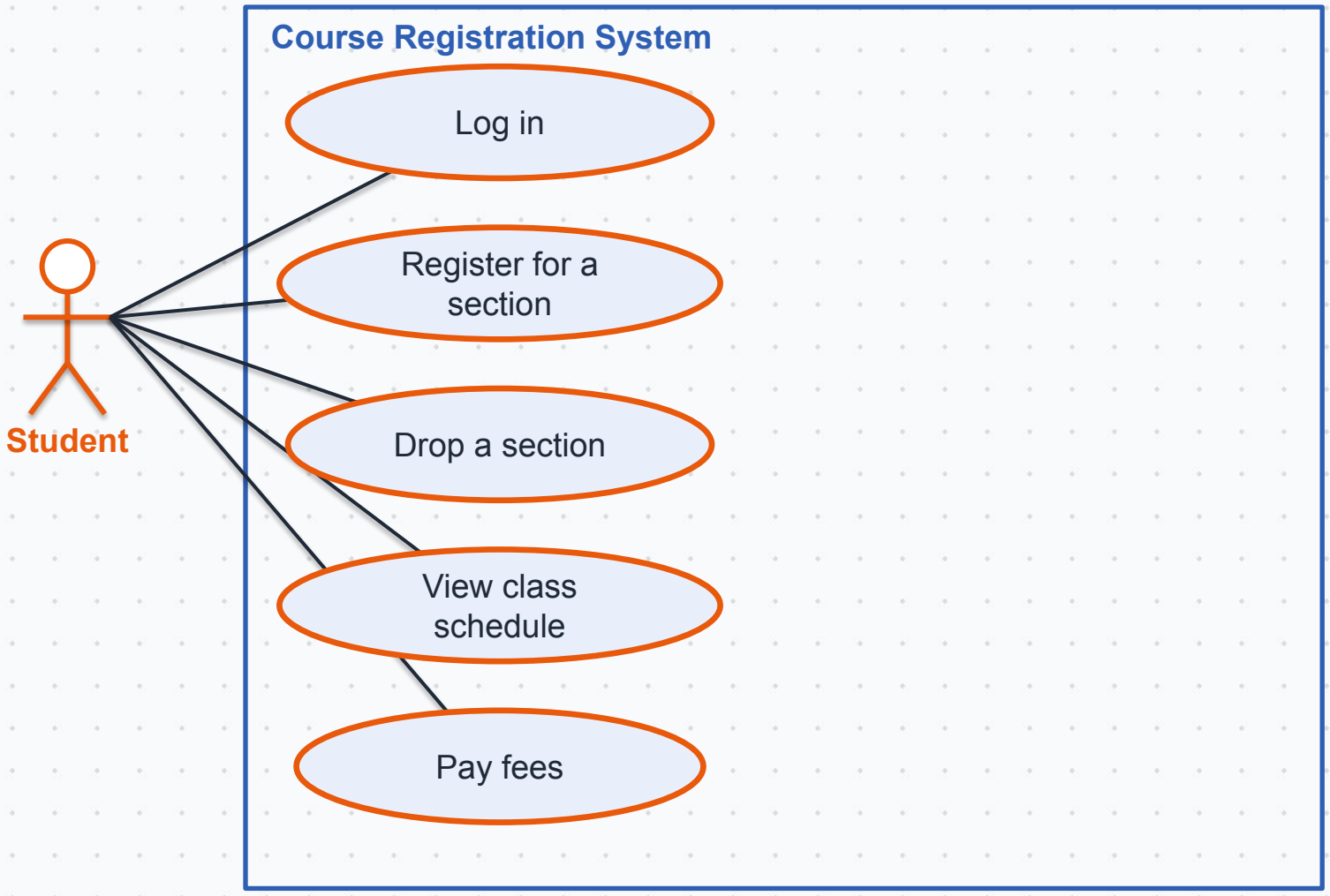
- The extension adds optional behaviour to the base.
- It runs only when its condition is true.
- **Arrow points to the base use case.**

Memory trick: the arrow always starts at the use case that “knows about” the other. Register knows it must check prerequisites; Join waitlist knows which use case it extends.

Five steps to build a use case diagram

	IN OUR EXAMPLE
1 Identify the actors	Student, Registrar staff, Student Records, Payment Gateway
2 Identify the use cases	Ask what each actor needs: log in, register, drop, view schedule, pay, manage sections
3 Look for common functionality → «include»	Registering always checks prerequisites
4 Can actors or use cases be generalized?	BSc and MSc students; card and mobile wallet payments
5 Find optional or additional functions → «extend»	Join waitlist, only when the section is full

Build it · steps 1–2: actor, boundary, goals

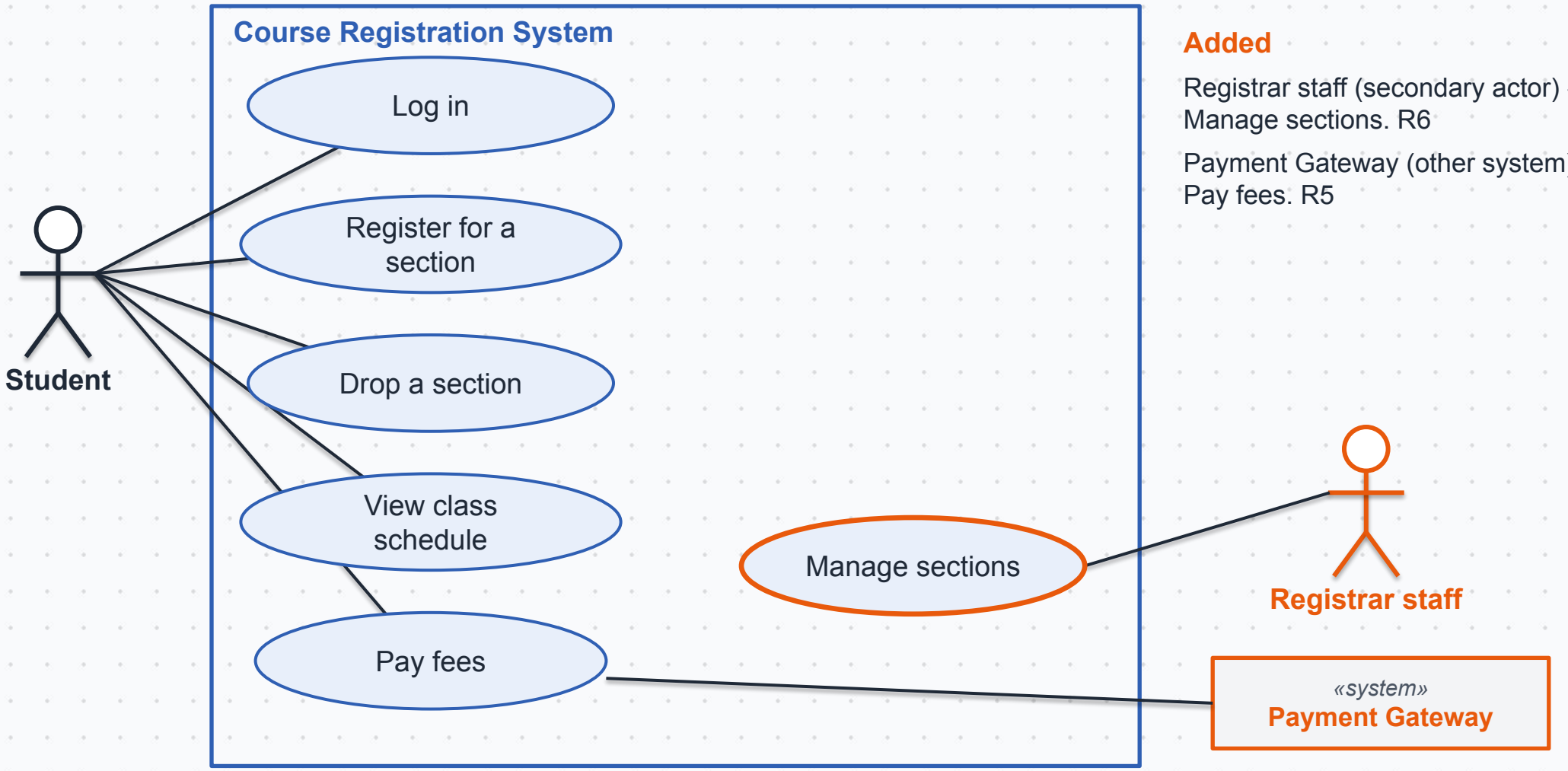


Actor
Student, from the story.

Use cases
One per student goal: R1, R2, R5.

Associations
Plain lines, no arrows.

Build it · add the other actors

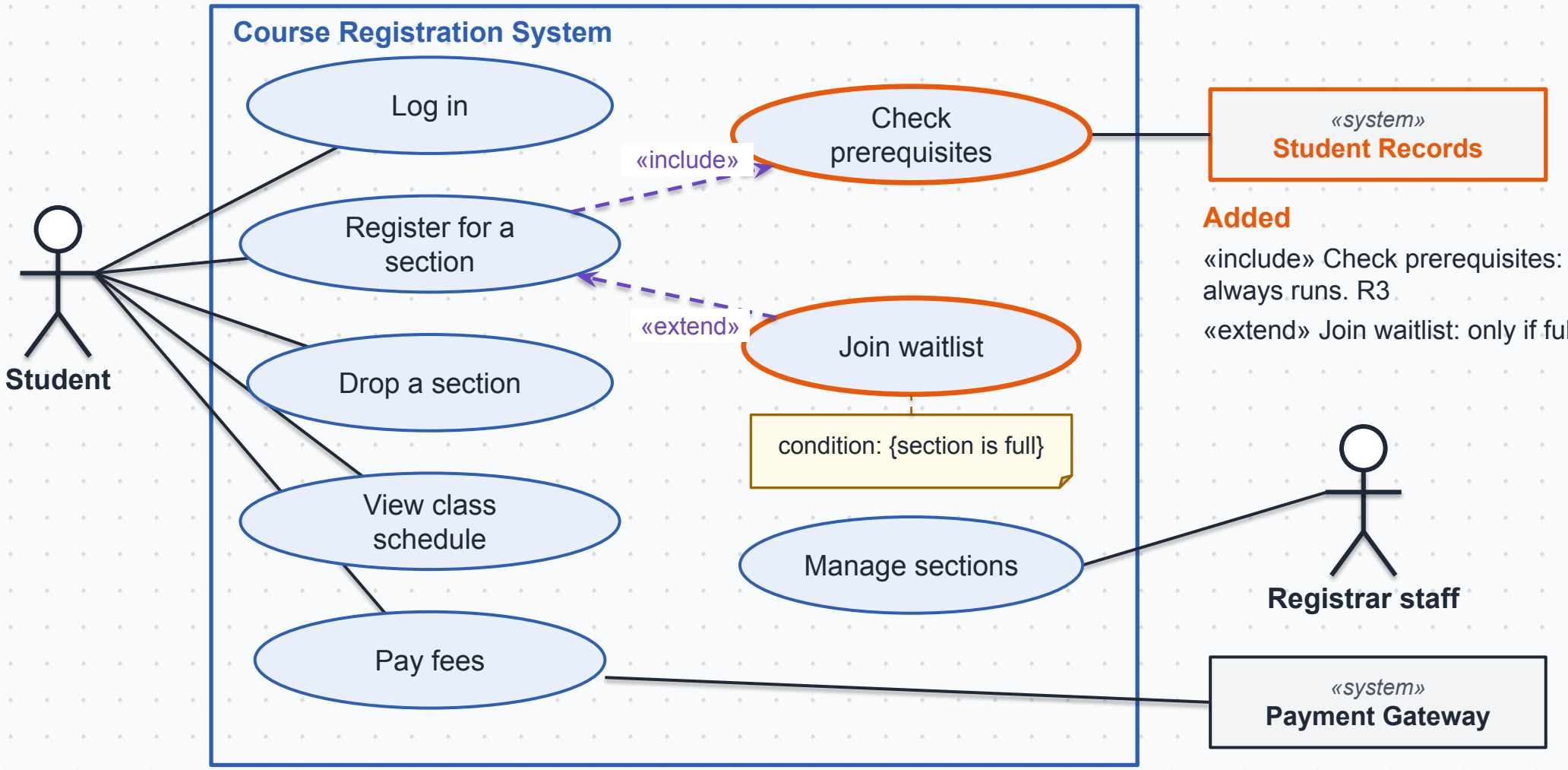


Added

Registrar staff (secondary actor) → Manage sections. R6

Payment Gateway (other system) → Pay fees. R5

Build it · include and extend



Added

«include» Check prerequisites: always runs. R3

«extend» Join waitlist: only if full. R4

What a use case diagram shows

It shows

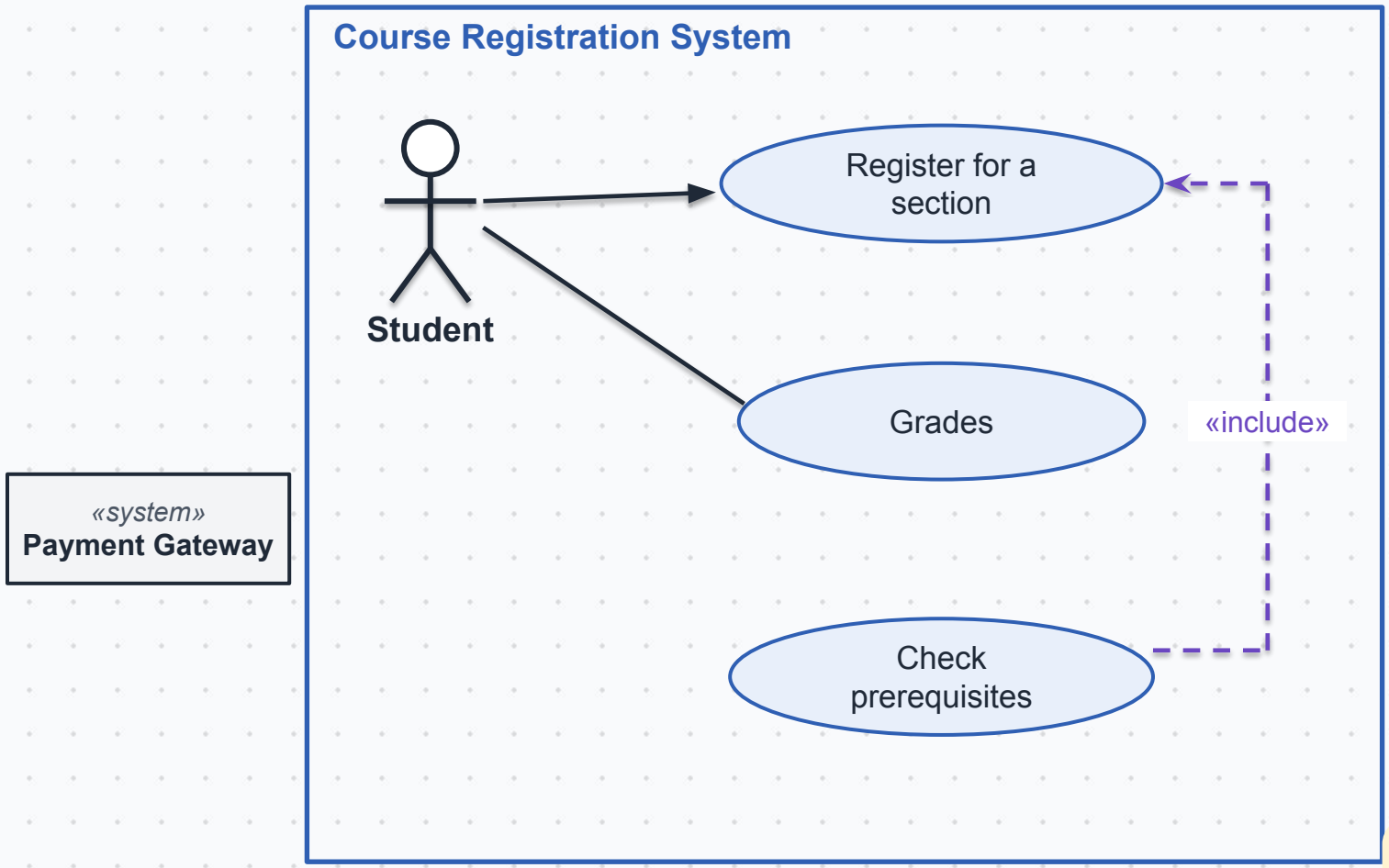
- ✓ Who uses the system: 2 people, 2 systems
- ✓ Which goals the system supports: 8 use cases
- ✓ What is inside and outside the system
- ✓ Reuse: registering always checks prerequisites
- ✓ Optional behaviour: waitlist, only when full

It does not show

- ✗ The order of the steps
- ✗ What happens when a check fails
- ✗ Which part of the system does what
- ✗ Screens, buttons or data fields

The use case description, and the sequence and activity diagrams, fill these gaps.

Which elements are wrong?

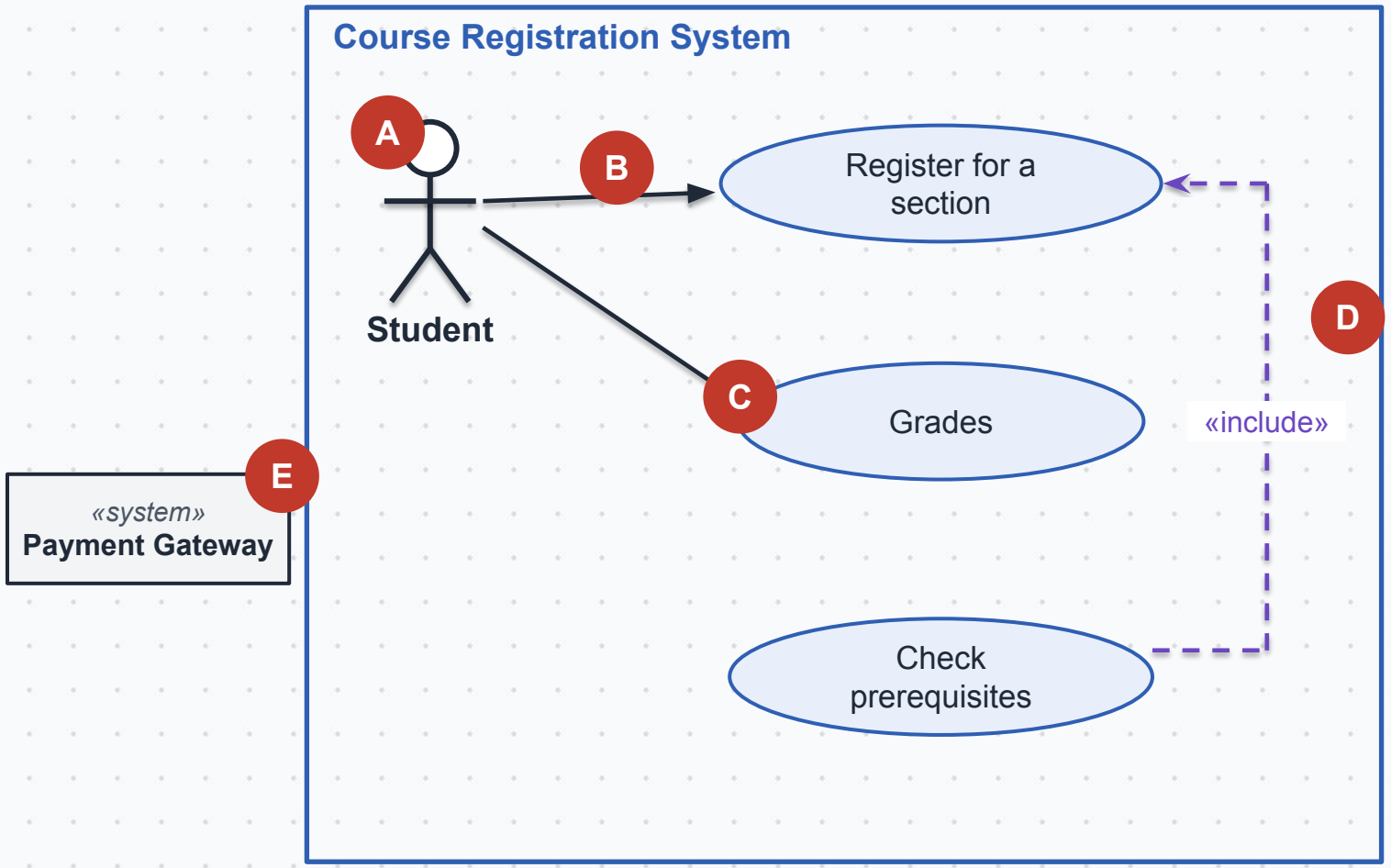


PRACTICE 2

This diagram has 5 mistakes.
Find each one and say how to fix it.

Think for 1 minute, then discuss with your neighbour.

Five mistakes, fixed



ANSWER 2

- **A** Student is inside the boundary. Actors go outside.
- **B** Association has an arrowhead. Use a plain line.
- **C** “Grades” is a noun. Use a verb: “View grades”.
- **D** Include arrow is reversed. Point it to Check prerequisites.
- **E** Payment Gateway has no association. Every actor needs one.

BRAC UNIVERSITY
Inspiring Excellence

Use case descriptions

Every use case needs a description, usually a table with a unique ID. Not every field is needed for every use case.

Identify it
Name · ID · Priority

Before
Preconditions: the state the system must be in

After
Postconditions: the results; they become preconditions of the next use case

Summary
Major inputs (with source) and outputs (with destination)

Who and when
Actor(s) · Description · Trigger (external or temporal)

The steps
Normal course (“happy path”) · Alternative courses

When things go wrong
Exceptions: errors that end without success

UC-02 Register for a section (1 of 2)

Use case name	Register for a section
ID · Priority	UC-02 · High
Actors	Student (primary) · Student Records system (other system)
Description	A student adds a course section to next semester's schedule. The system checks prerequisites and seat availability, then confirms or explains why it failed.
Trigger	The student selects a section to add. Type: external
Preconditions	1. The student is logged in (UC-01 Log in). 2. The registration period is open. 3. The section list is up to date.

UC-02 Register for a section (2 of 2)

Normal course (happy path)	1. The student selects a section. 2. The system checks prerequisites with Student Records (include: Check prerequisites). 3. The system checks that a seat is free. 4. The system reserves the seat and records the enrollment. 5. The system shows a confirmation and the updated schedule.
Alternative course	3a. The section is full: the system suggests another open section of the same course; the student picks it; continue at step 3. (Or the student joins the waitlist: extend.)
Exceptions	E1 (step 2): a prerequisite is missing → show which course is missing; the use case ends without enrollment. E2 (step 4): another student took the last seat a moment earlier → show “section just filled”; the use case ends.
Postconditions	Enrollment recorded · seat count reduced by 1 · fees recalculated (a precondition for UC-05 Pay fees).
Summary inputs → outputs	In: section choice (Student), prerequisite status (Student Records). Out: confirmation or reason (Student), new fee amount (Pay fees).

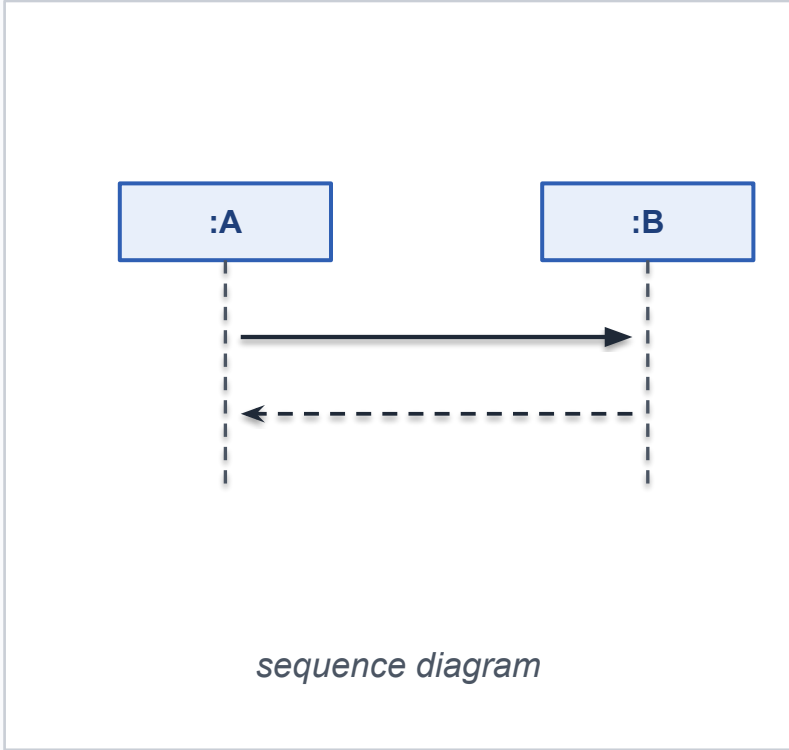
BRAC UNIVERSITY
Inspiring Excellence

02

PART 2

Sequence diagrams

In what order do the parts of the system talk to each other?



sequence diagram

Parts of a sequence diagram



Activation bar
busy working

Lifeline
one participant; time runs down



Synchronous message
the sender asks and waits



Return message
the reply, dashed



Self-message
a participant calls itself



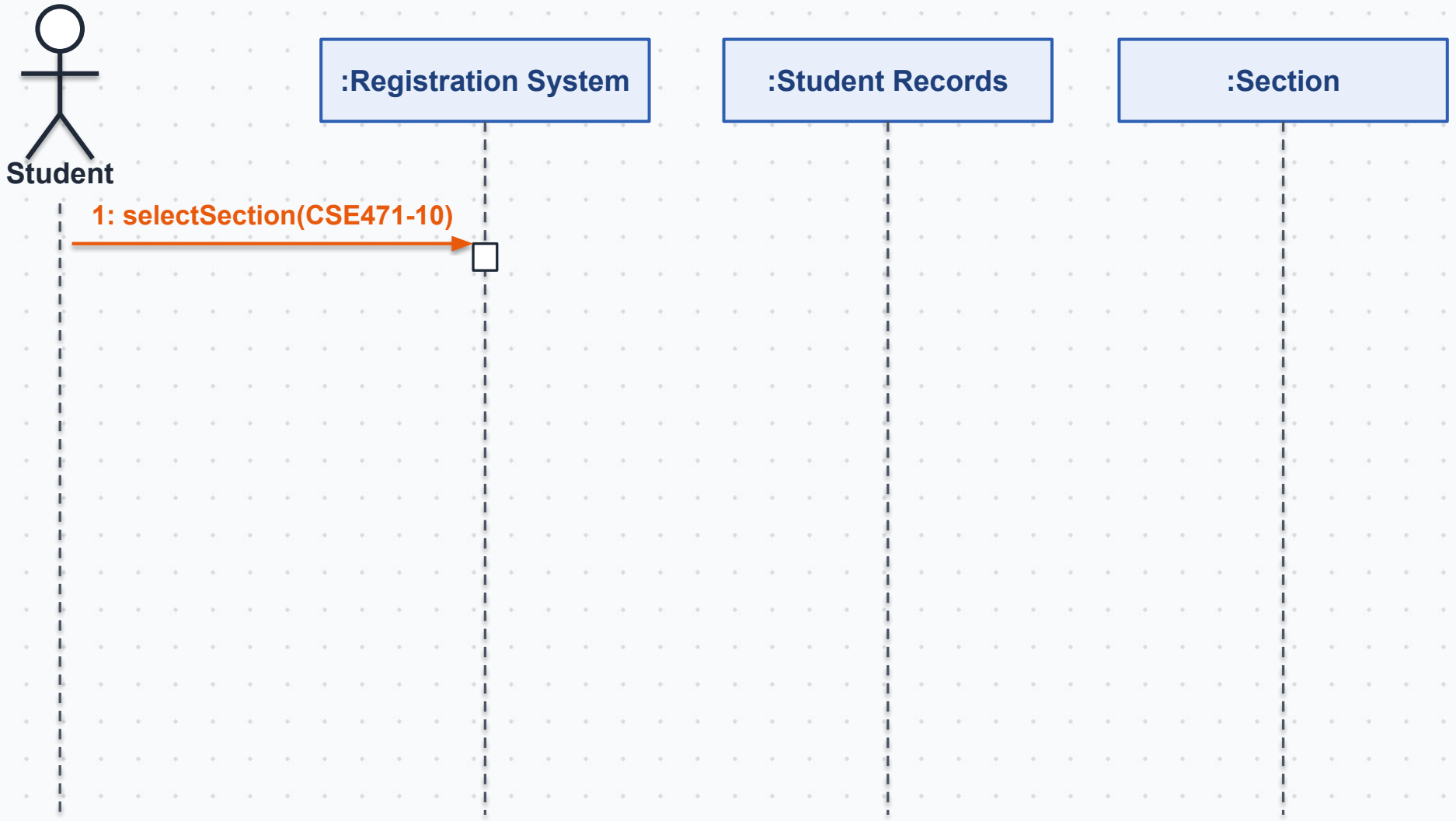
alt fragment
one section runs, chosen by its [guard]

Read it

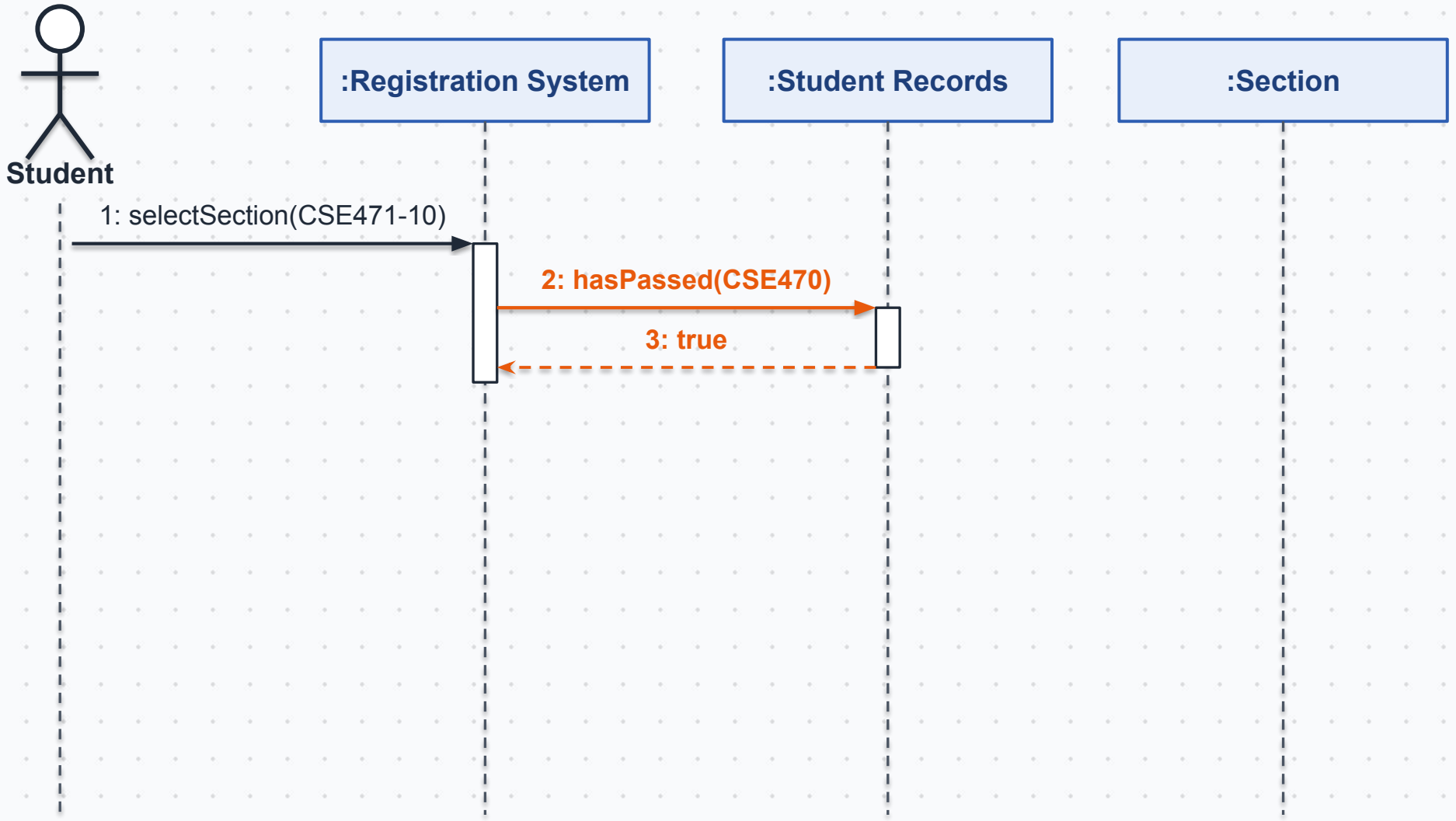
top to bottom = earlier to later

left to right = who talks to whom

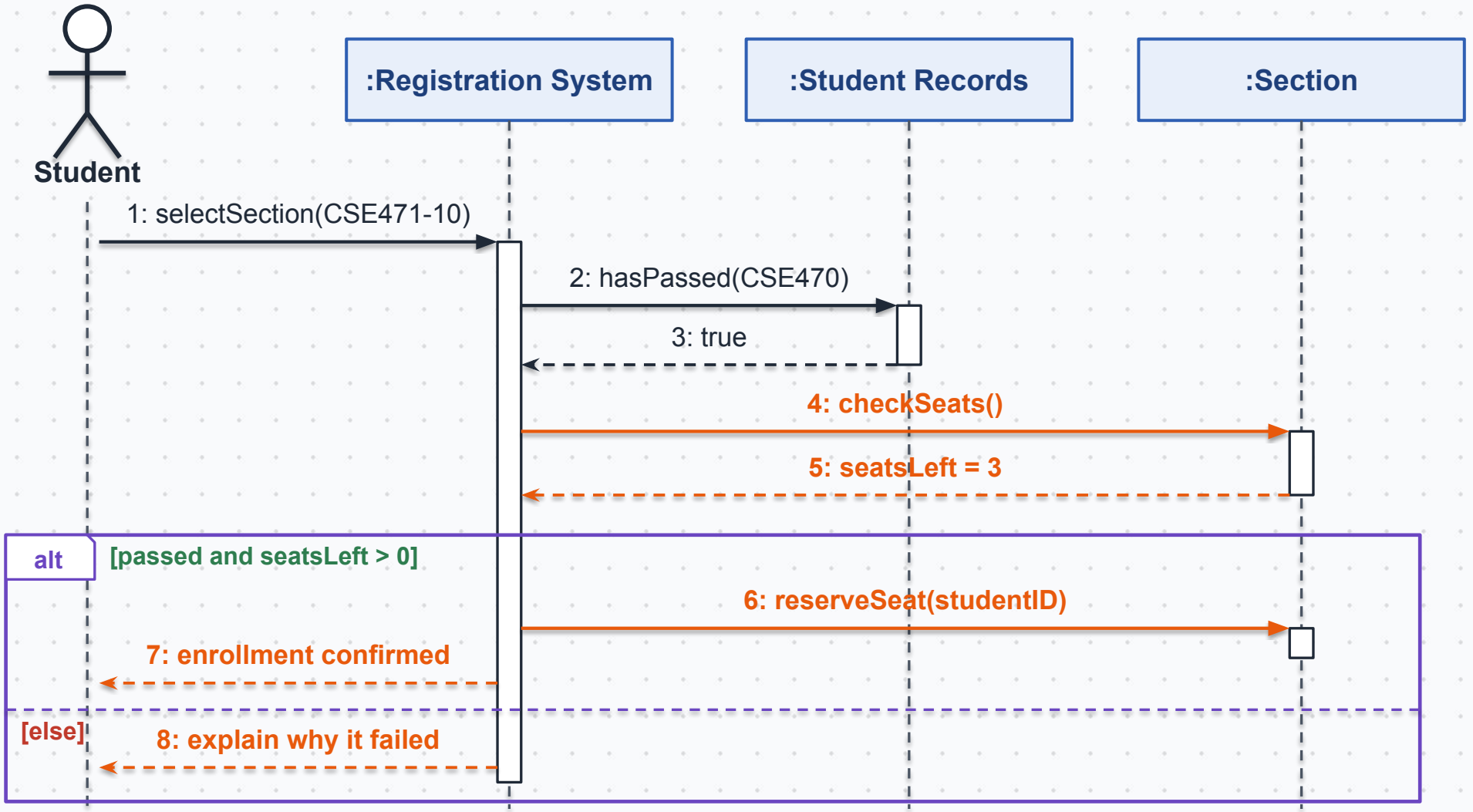
Build it · the student starts the use case



Build it · check prerequisites



Build it · check seats, then confirm or explain



Reading the sequence diagram

It shows

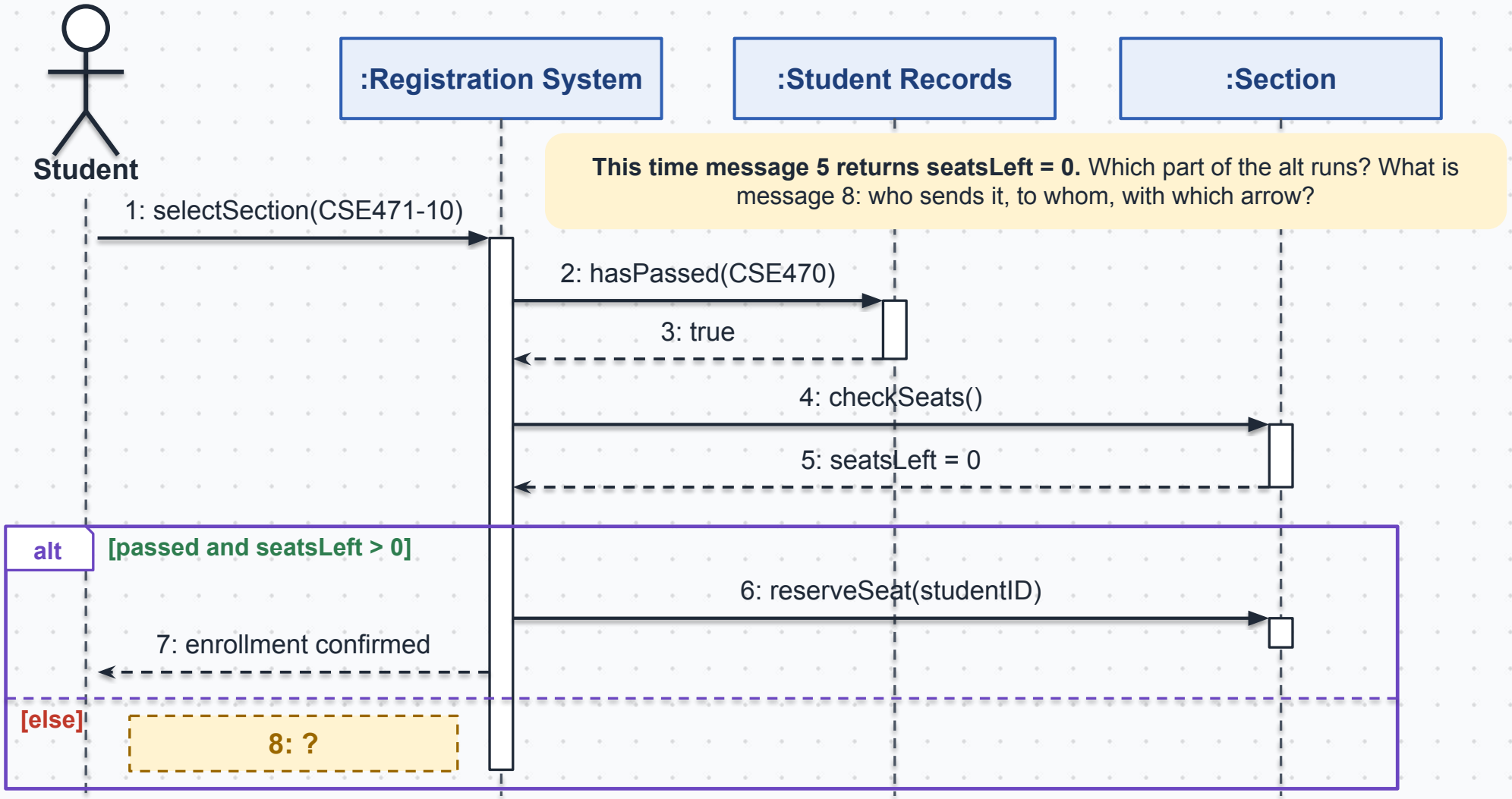
- ✓ Which participants take part in one use case
- ✓ The exact order of messages, numbered 1 to 8
- ✓ Who waits for whom (activation bars)
- ✓ Two outcomes in one picture (alt)

It does not show

- ✗ Other use cases (one diagram per scenario)
- ✗ Work that happens in parallel, clearly
- ✗ Long, branching workflows: these get crowded fast

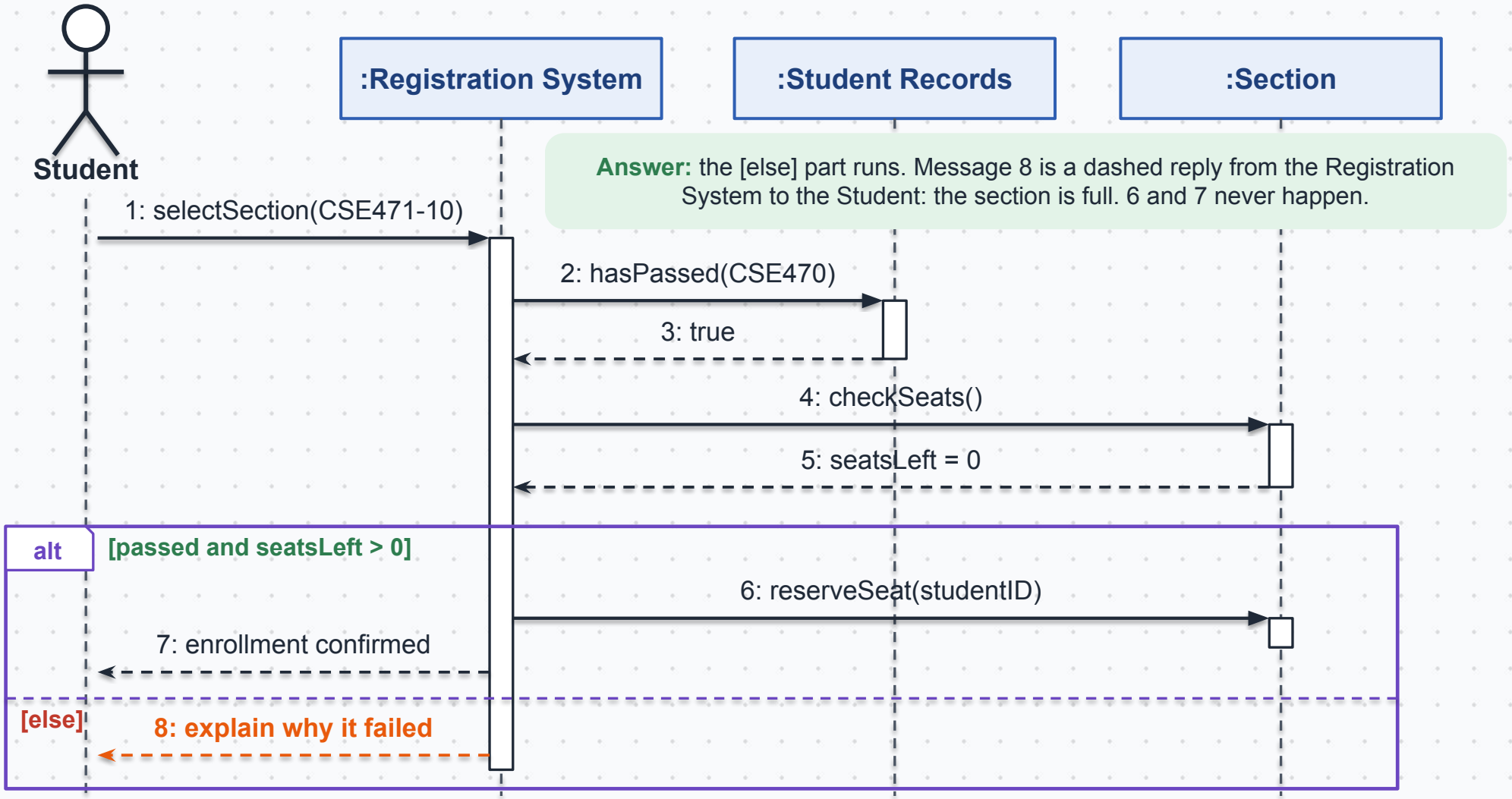
Trace it to the description: messages 1–7 are the normal course; message 8 is exception E1.

What happens next?



ANSWER 3

What happens next?



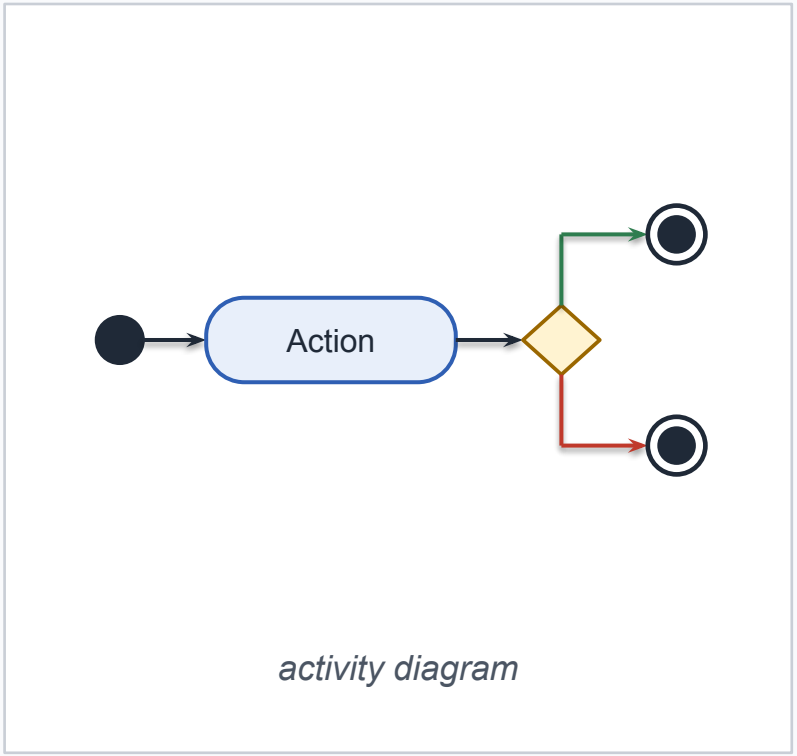
BRAC UNIVERSITY
Inspiring Excellence

03

PART 3

Activity diagrams

What are the steps, decisions and parallel paths?



activity diagram

What is an activity diagram?

- **A behaviour diagram** that shows the sequence and conditions of the steps in a process.
- It focuses on the flow of work, not on which object owns each step.
- Also called **control-flow and object-flow models.**
- A step can start because another finishes, because data becomes available, or because an outside event happens.
- Shows **sequential and parallel** activities.

Useful for modelling

- Business processes
- Workflows
- Data flows
- Complex algorithms

Start, end and actions

Initial node
Where the flow starts when the activity begins. An activity may have more than one.

Activity final node
An activity may have several. The first one reached stops all flows.

Check prerequisites

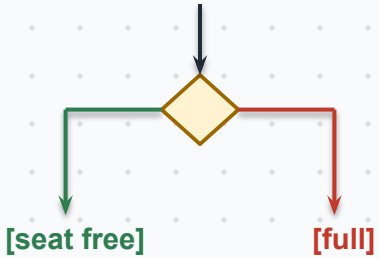
Action
One step that is not broken down further in this diagram.

Simple from the diagram's point of view, though it may be complex inside.

Starts only when all its inputs are ready.

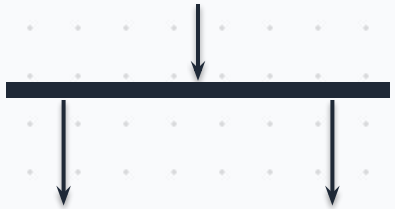
An activity is made of actions, and can be reused in many places.

Decision, merge, fork and join



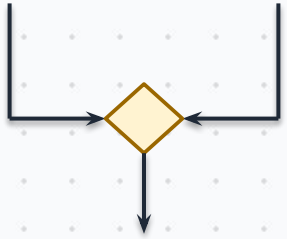
Decision

One way in, several out. A [guard] on each exit picks the path.



Fork

One flow in, several parallel flows out. All branches run.



Merge

Several alternative flows in, one out. Accepts whichever arrives; does not wait.

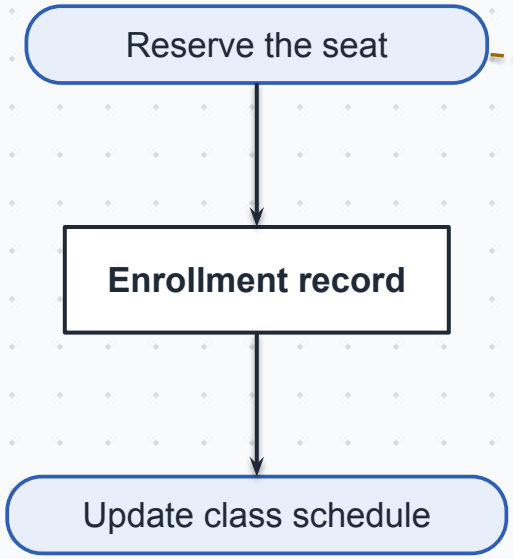


Join

Several parallel flows in, one out. Waits for all of them.

Don't mix them up: a diamond chooses ONE path (decision) or accepts ONE of several (merge). A bar runs ALL paths at once (fork) or waits for ALL (join).

Object nodes and notes

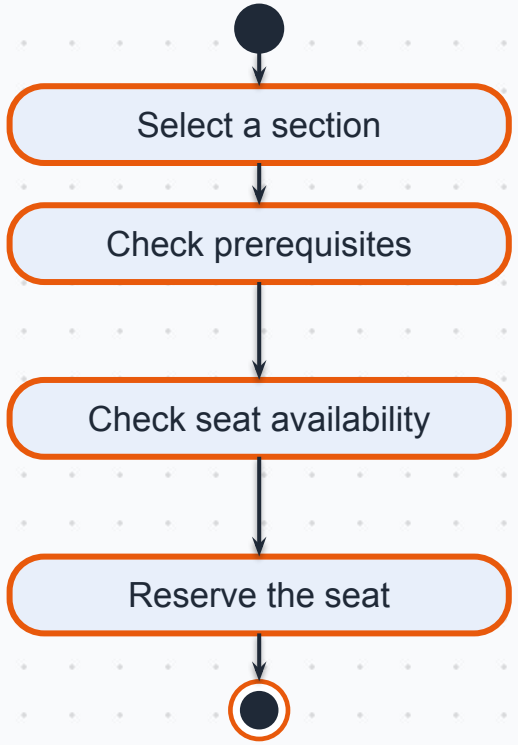


Object node
A thing (an instance of a class, maybe in a given state) available at that point in the flow.

Seat count is read live, not from yesterday's list.

Note (comment)
Attaches a remark to any element. It has no meaning in the model, but helps the reader.

Build it · the happy path



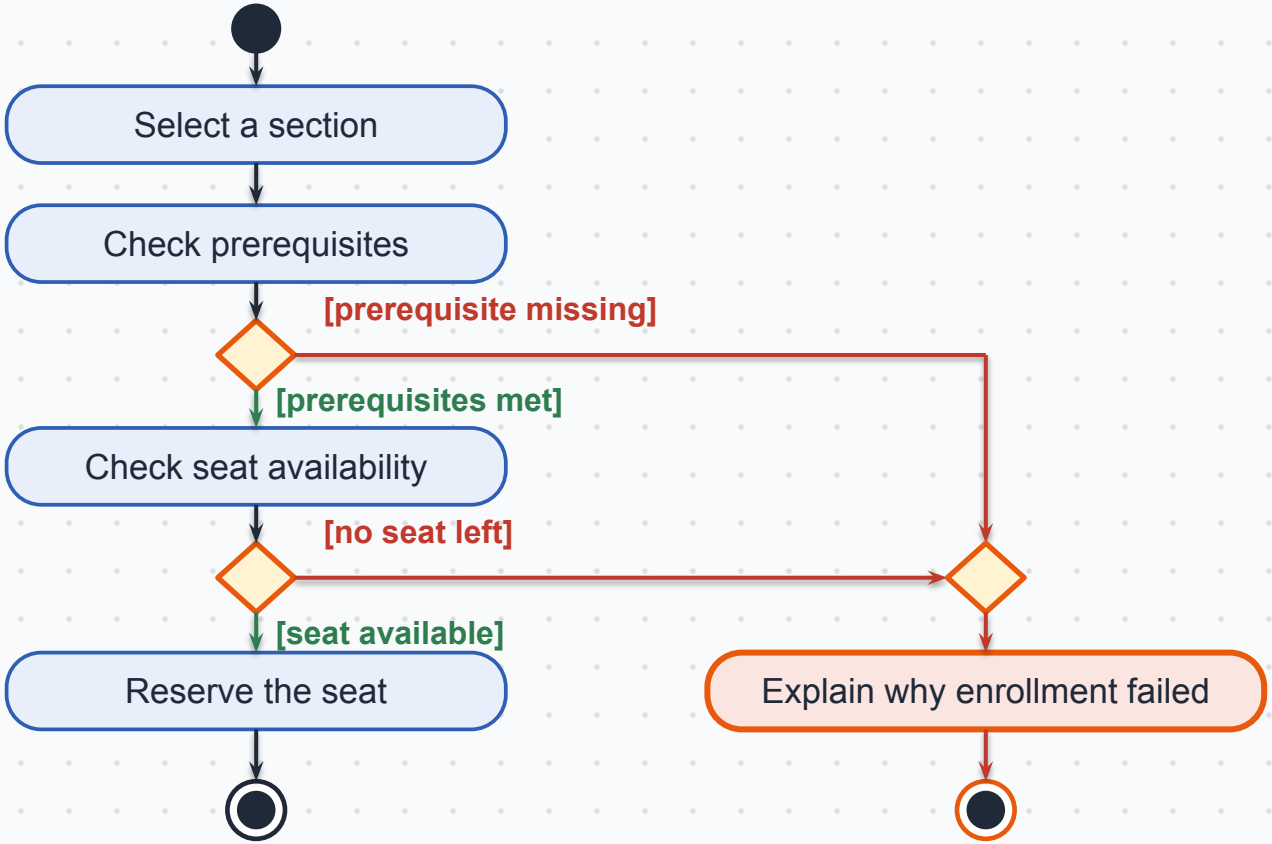
From the normal course

Steps 1–4 of UC-02 become four actions, in order.

Start → actions → end.

Ask: what if a check fails?

Build it · decisions and the failure path

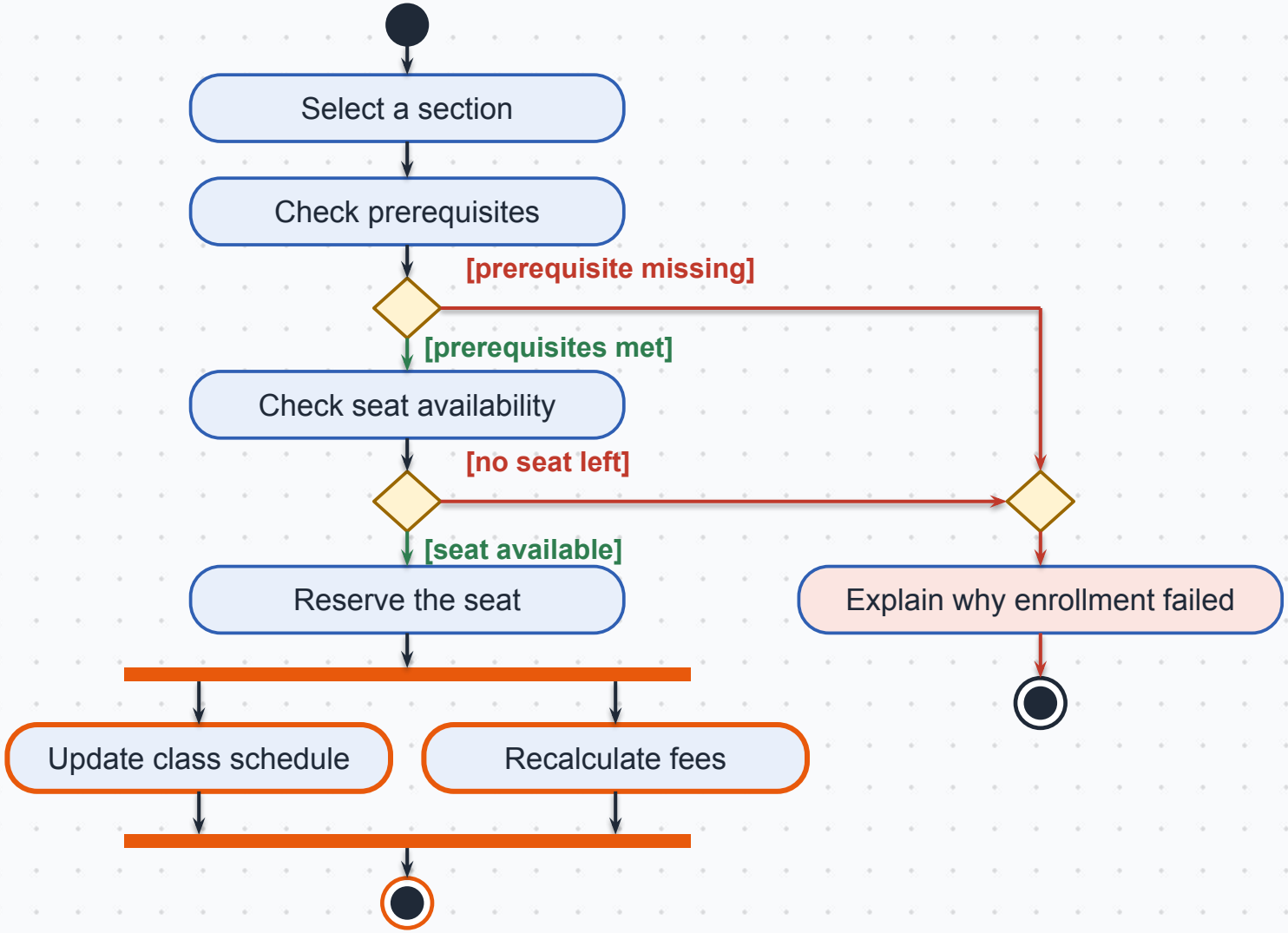


Decisions
Each check is followed by a diamond with [guards].

Merge
Both failures share one path.

Two final nodes
One for success, one for failure.

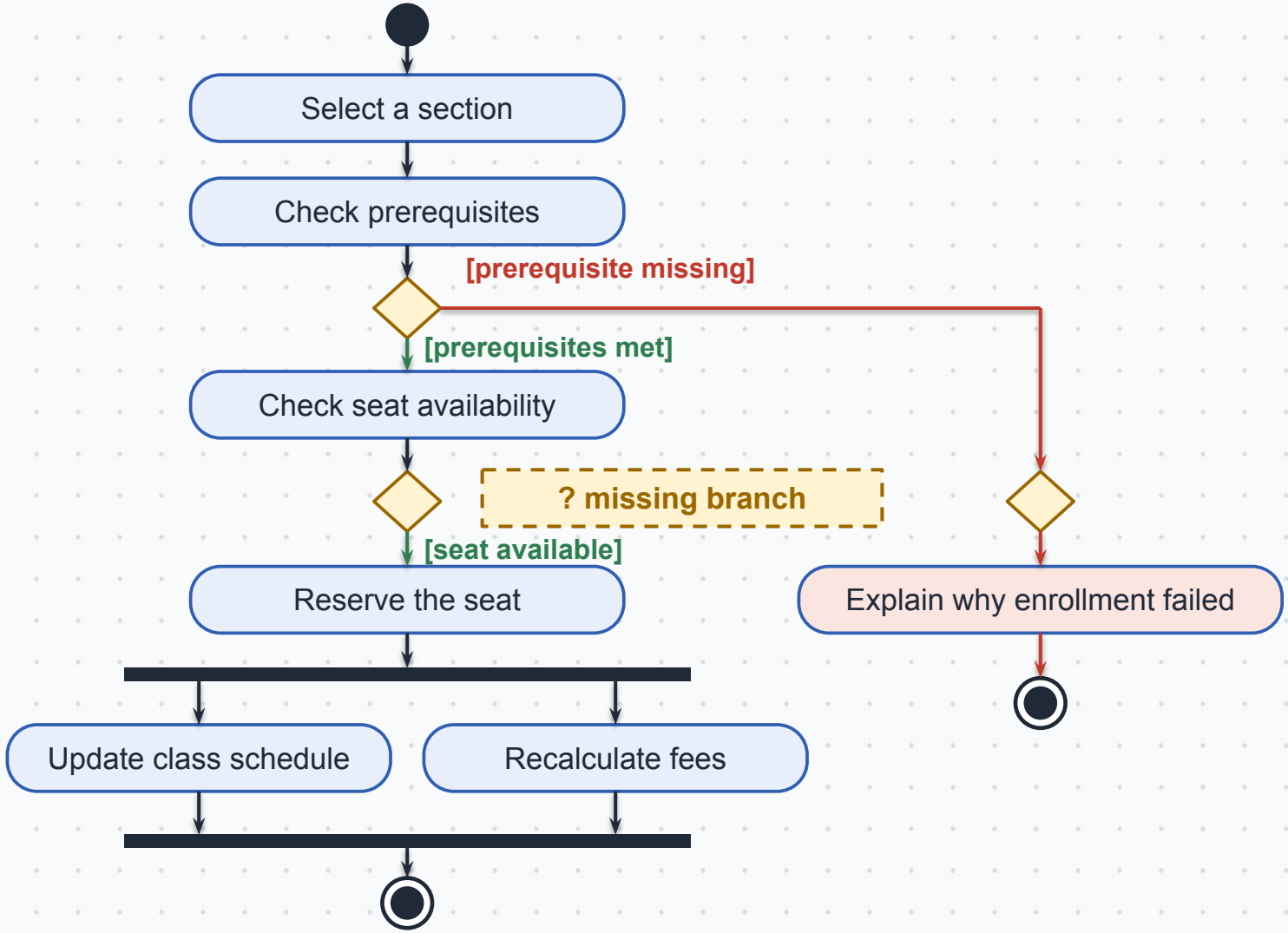
Build it · parallel work with fork and join



Fork
After reserving, two things happen at once.

Join
The activity ends only when both are done.

Complete the missing branch



PRACTICE 4

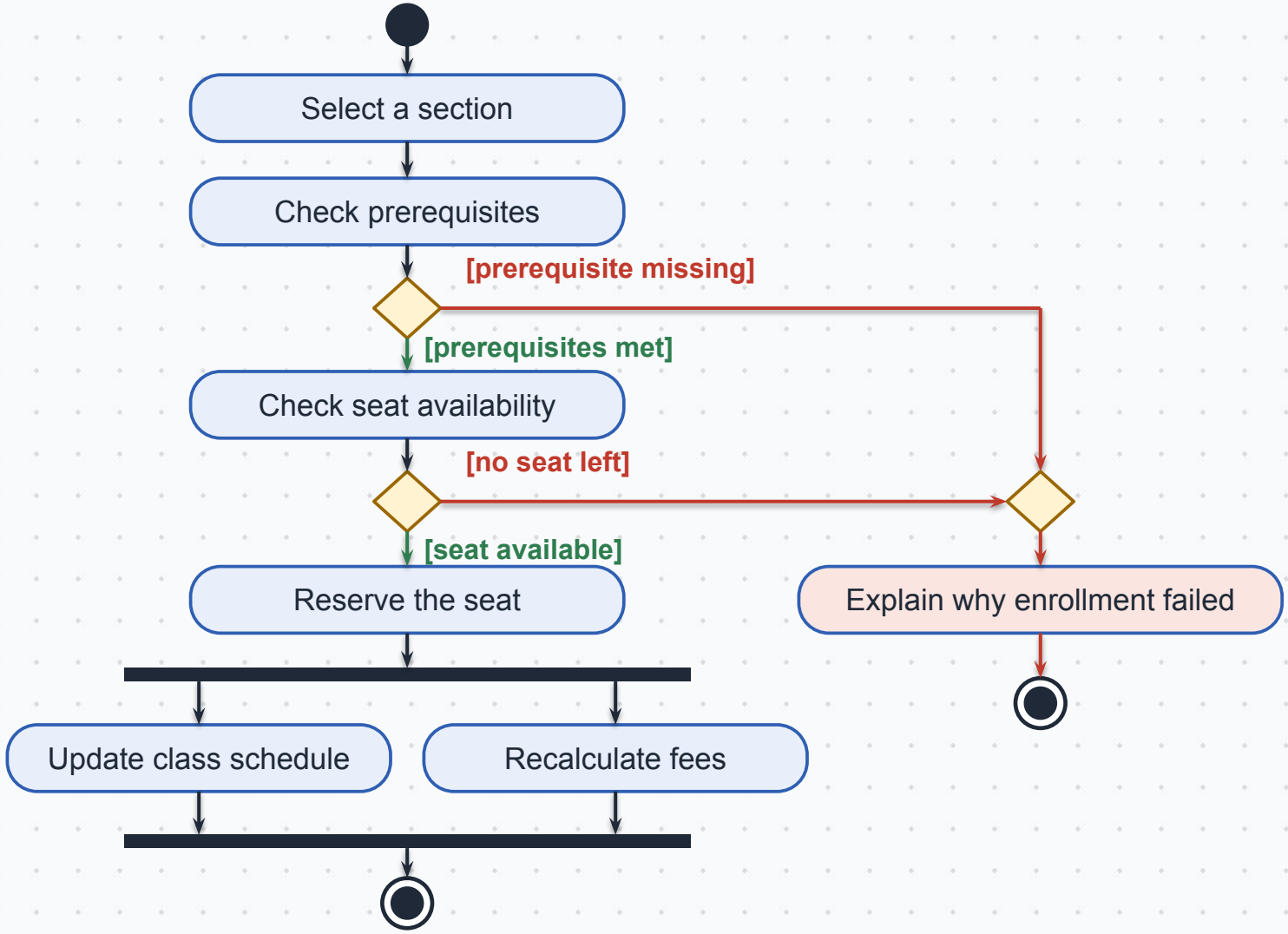
One flow is missing.

Which decision is it on?

Where does it go?

What is its guard?

Complete the missing branch



ANSWER 4

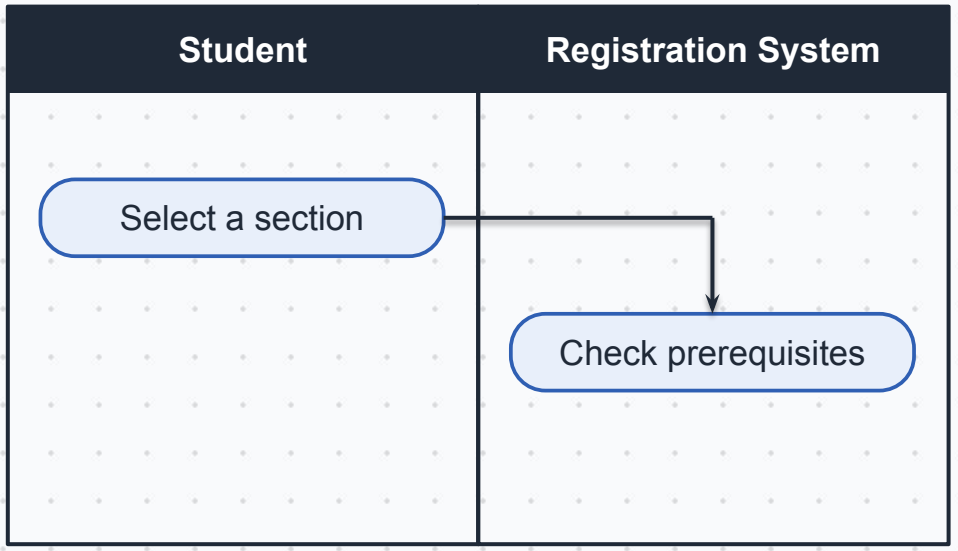
Answer

From the seat decision, guard **[no seat left]**, flowing into the merge.

Why: every decision needs an exit for every possible outcome, and the guards must not overlap.

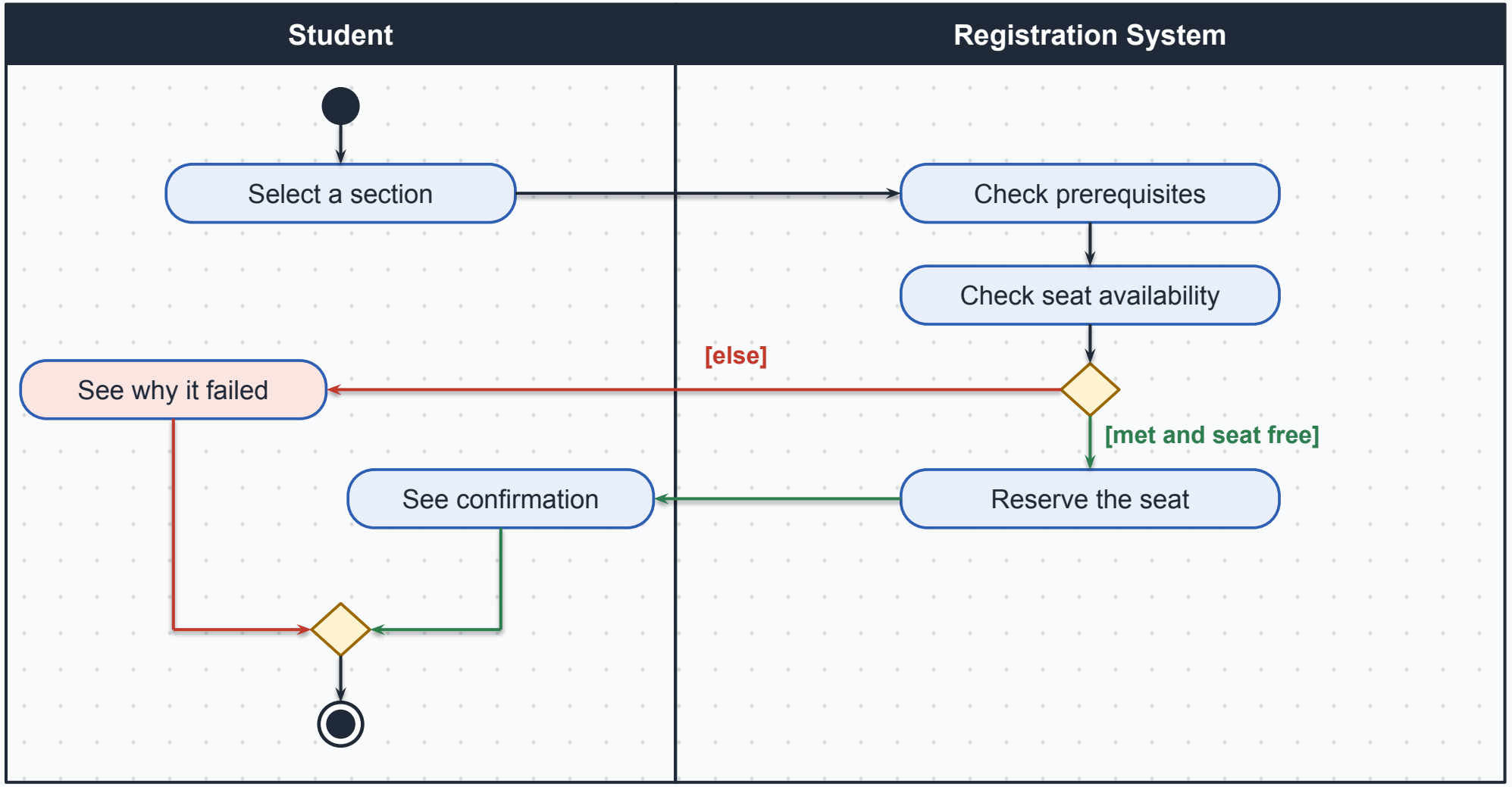
Swimlanes: who does each step?

- Swimlanes (activity partitions) show where activities take place.
- They can also show areas at the technology level, such as the browser and the server.
- Each action sits in the lane of whoever performs it.
- Partition by **actor or class**: can reveal links missing from the class model.
- Partition by **use case**: documents how use cases interact.



Two lanes, one per actor. A flow that crosses a lane line means work passes from one party to another.

Registration with swimlanes



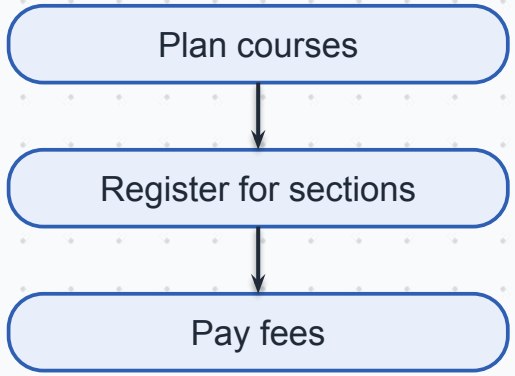
Guidelines for activity modelling

1. **Use it where it pays off**
   Most valuable for complex processes involving many parties.

2. **Start high, then zoom in**
   A “level 0” overview with only high-level actions, then a “level 1” diagram for each, and so on.

3. **Keep the level of detail even**
   Don't put “Deliver order” next to “Calculate tax” in the same diagram.

LEVEL 0: enrollment overview



LEVEL 1
“Register for sections” expands into the diagram we just built.

BRAC UNIVERSITY
Inspiring Excellence

04

PART 4

Compare, practise, recap

Which diagram answers which question?

One use case, three views

Use case diagram	Sequence diagram	Activity diagram
Who uses the system, and for what?	***Who talks to whom, in what order?***	***What are the steps, choices and parallel paths?***
• Actors and their goals • System scope (boundary) • Reuse and options: include, extend	• Participants (lifelines) • Numbered messages over time • Alternatives with alt	• Actions and control flow • Decisions, merges, forks, joins • Who does what (swimlanes)
Limit: No order, no logic	**Limit:** Gets crowded for long workflows	**Limit:** Hides which object sends which message

Which diagram best answers each question?

PRACTICE 5

1. Does the Student Records system talk to the registration system before or after the seat check?
2. Which goals can registrar staff achieve with the system?
3. What happens when a student has no seat available and no missing prerequisite?
4. Is the Payment Gateway part of what we are building?

Choose: use case · sequence · activity. Say why.

Think for 1 minute, then discuss with your neighbour.

Which diagram best answers each question?

ANSWER 5

1. **Sequence** — Order of messages between participants is exactly what it shows (messages 2–3 come before 4–5).

2. **Use case** — Goals per actor: Registrar staff → Manage sections.

3. **Activity** — Follow the decisions: [no seat left] → merge → Explain why enrollment failed. The sequence diagram's [else] works too, but hides which check failed.

4. **Use case** — The system boundary shows it is outside: we use it, we do not build it.

Final practice: library study-room booking

PRACTICE 6

Students log in and book a library study room for one hour.

The system checks that the student is not banned for past no-shows, and that the room is free at that hour.

If both checks pass, it books the room and sends a confirmation. Otherwise it explains why and suggests another time.

For a group booking, the student may also invite group members; invitations go out through the university email service.

Library staff can block a room for maintenance.

a Draw the use case diagram, with one «include» and one «extend».

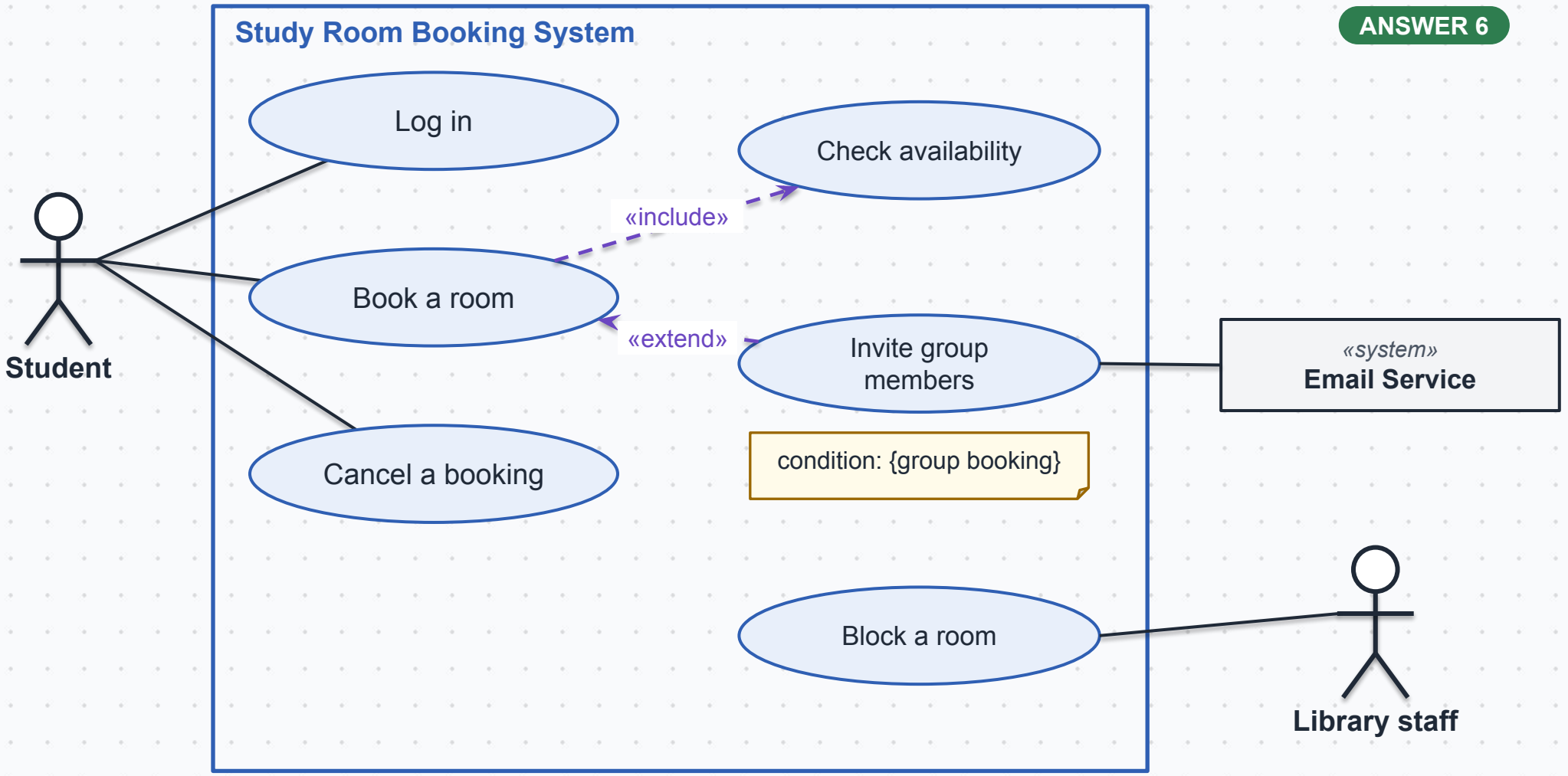
b Sequence diagram for “Book a room”: name the lifelines and the messages, with an alt.

c Activity diagram for “Book a room”, with two decisions and a merge.

Work in pairs: 10 minutes.

Answer (a): use case diagram

ANSWER 6



Answers (b) and (c): sequence and activity

(b) Sequence: Book a room

Lifelines: Student · :Booking System · :Student Records · :Room

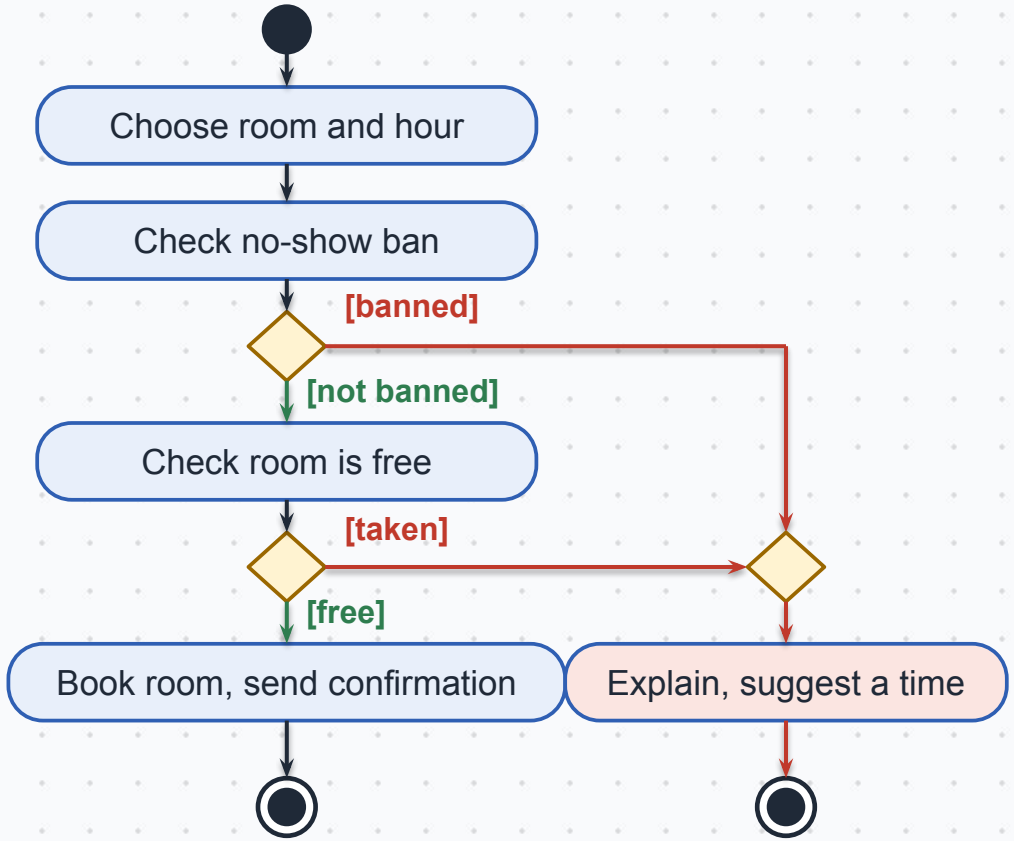
1. Student → Booking System: bookRoom(room, hour)
2. Booking System → Student Records: checkBan(studentID)
3. Student Records → Booking System (reply): notBanned
4. Booking System → Room: isFree(hour)
5. Room → Booking System (reply): free = true

alt [not banned and free]: 6 reserve(hour) → Room; 7 confirmation → Student (reply)

[else]: 8 reason and another time → Student (reply)

(c) Activity: Book a room

ANSWER 6



BRAC UNIVERSITY
Inspiring Excellence

Recap

Use case
One goal an actor reaches with the system. Verb-first name, ellipse, inside the boundary.

Use case diagram
Actors outside, use cases inside. Association, generalization, «include» (always), «extend» (sometimes).

Use case description
Trigger, preconditions, normal course, alternatives, exceptions, postconditions.

Sequence diagram
Lifelines and numbered messages over time; alt for if/else.

Activity diagram
Start, actions, decisions with guards, merge, fork/join, final; swimlanes for who.

Choosing
Scope → use case. Message order → sequence. Steps and choices → activity.

BRAC UNIVERSITY
Inspiring Excellence

Hospital reception

Purpose: describe the major services provided by a hospital's reception.

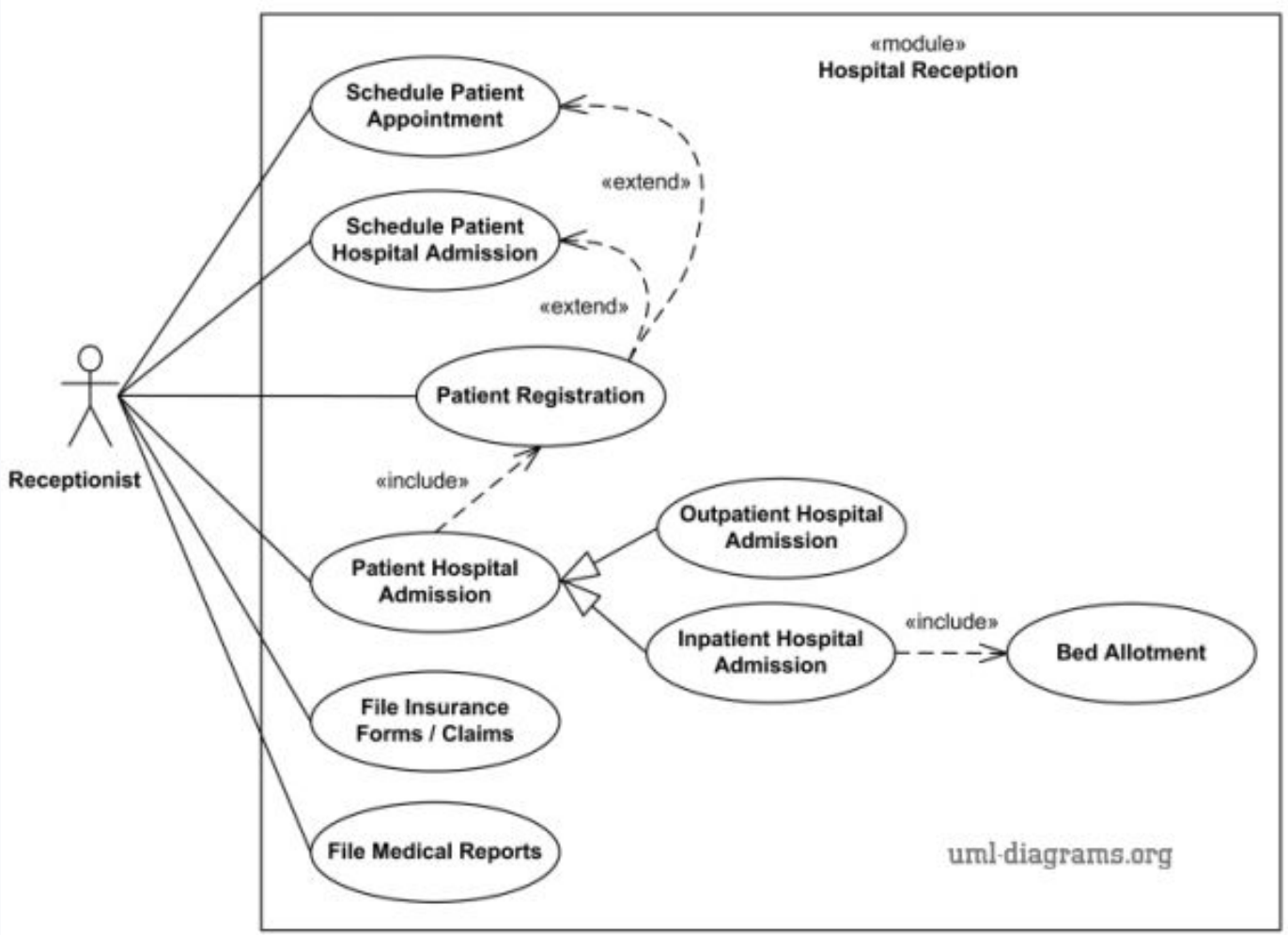
The Hospital Reception subsystem supports some of the receptionist's duties. The receptionist schedules patients' appointments and admission to the hospital, and collects information from the patient on arrival or by phone.

A patient who will stay in the hospital (an inpatient) should have a bed allotted in a ward.

Receptionists might also receive patients' payments, record them in a database and provide receipts, file insurance claims and medical reports.

Task: draw the use case diagram for the reception subsystem.

Hospital reception: model answer



Read it

One actor, the Receptionist.

Patient Hospital Admission is generalized into outpatient and inpatient admission.

Inpatient admission includes Bed Allotment.

Source: uml-diagrams.org, as used in the original deck.

Exercise 1 · ATM

USE CASE DIAGRAM

A bank customer must provide a PIN to log in, which the bank verifies. The customer may enter the PIN wrongly; an error message is then shown. The customer can make transactions: fund transfer, withdrawal, PIN change, balance check and deposit. The ATM prints a receipt if the customer wants one, for a charge. Charges also apply to each transaction. A system administrator monitors users' transactions and reports any suspicious activity.

Hint for the instructor: Look for: an «extend» for the receipt, an «include» for charges, and the administrator as a second actor.

Exercise 2 · Course management

USE CASE DIAGRAM

Two types of students use the system, BSc and MSc. Students log in with a username and password; the system checks the credentials and shows an error if they are wrong. Students add courses for next semester; the system checks remaining seats and completed prerequisites, and might suggest additional courses. After adding courses, a student may request advising, which an adviser approves. Students can view their routine and grade sheet. BSc students add project proposals and MSc students thesis proposals, approved by the advisor. Students pay by card or bank transfer; the system verifies the payment.

Hint for the instructor: Look for: actor generalization (BSc, MSc), use case generalization (card, bank transfer), includes for the checks, extend for suggestions.

Exercise 3 · Courier service

USE CASE DIAGRAM

A customer registers or logs in. To place an order the customer must be authenticated, then enters shipment details, chooses delivery preferences and pays. The order is confirmed only after payment through an online gateway; the system then generates a courier slip with tracking information. Courier staff see new orders and schedule pickups. The admin monitors operations, manages users and branch logistics, and handles complaints. Customers track shipments in real time and may cancel an order not yet dispatched. Delivery agents update the parcel's status at each stage and confirm delivery by OTP or signature.

Hint for the instructor: Look for: four human actors plus the payment gateway; cancel is only possible before dispatch.

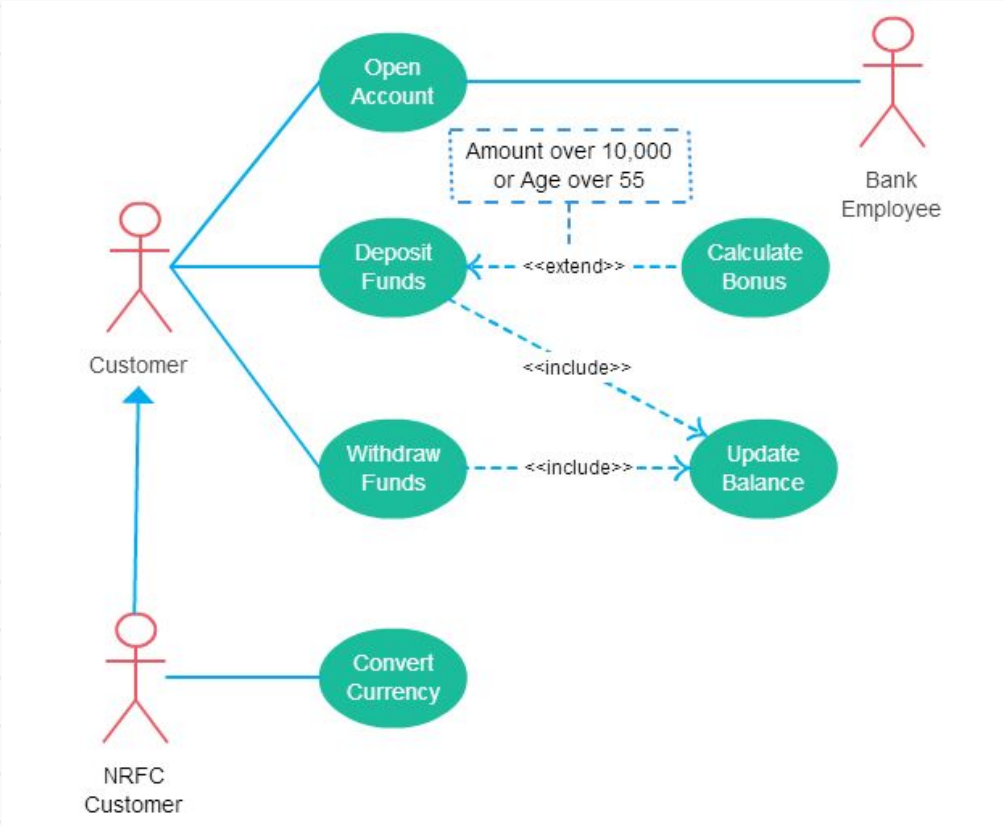
SAF authorization

ACTIVITY DIAGRAM

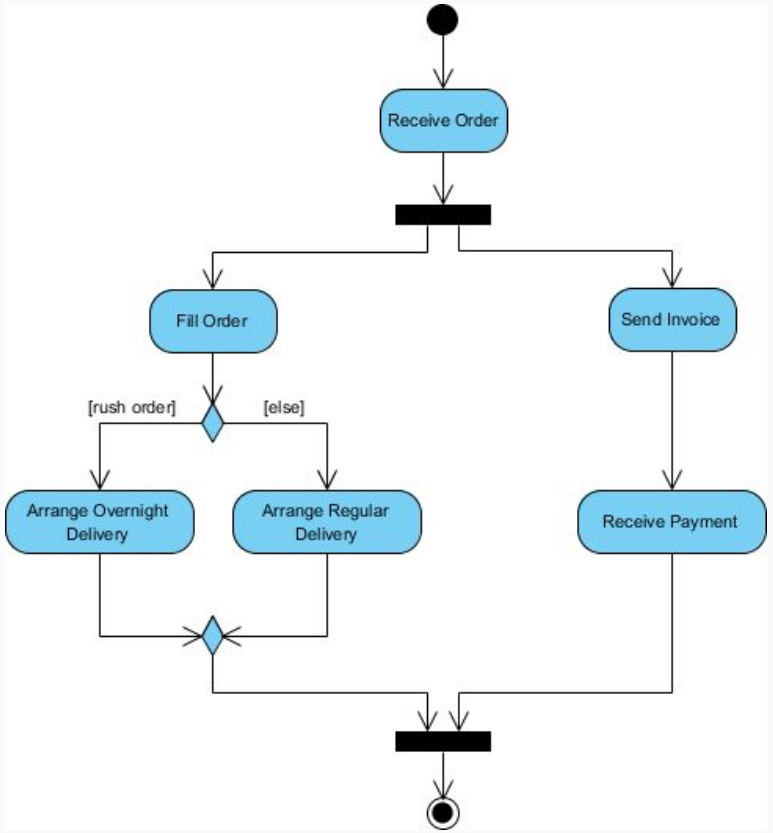
A student fills out a Student Assistance Fund form and sends it to the departmental chairperson. If the amount is small (under Tk 10,000), the chairperson signs it and routes it to accounts payable, which enters it in the accounting system. The system cuts a cheque for the student; once cashed, the form is filed with the cancelled cheque. If the cheque is not cashed within 30 days, the form expires. For large amounts (over Tk 10,000), the chairperson signs and sends the form, with a paragraph of reasons, to the chief financial officer, who signs and passes it to accounts payable. Either signer may reject the form; the student can then add more explanation or pay the whole fee.

Hint for the instructor: Look for: swimlanes per role, a decision on the amount, a decision on reject or approve, and the 30-day expiry.

Worked examples from the original deck

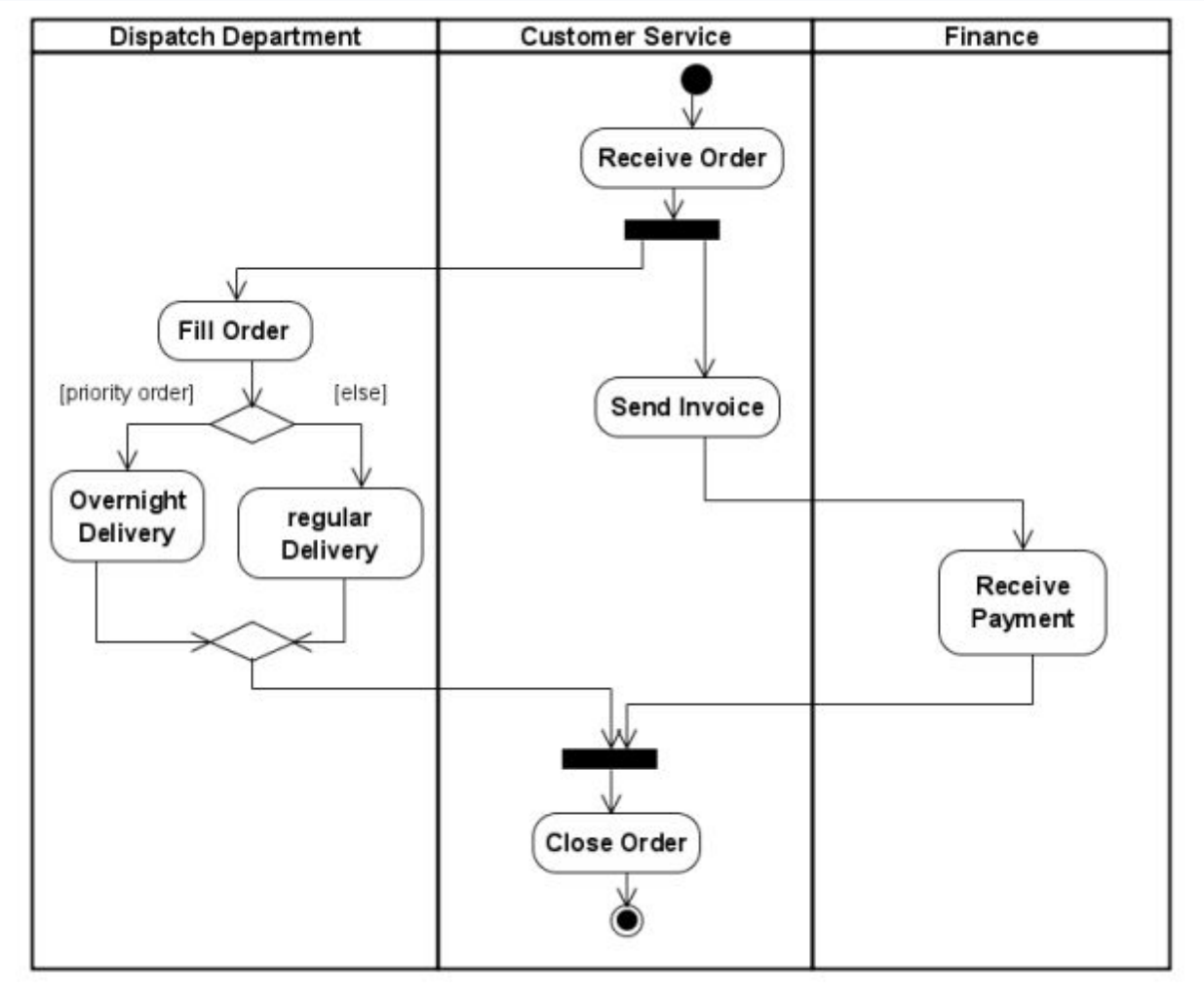


Bank: generalization (NFRC customer is a kind of customer), «include» Update balance, «extend» Calculate bonus.



Process Order: fork into fill-order and billing, a decision for delivery type, then a join to close the order.

Swimlane example from the original deck



HIDDEN SLIDE · NOT SHOWN IN SLIDESHOW

Instructor notes: changes from the original deck

BRAC UNIVERSITY
Inspiring Excellence

- **Sequence diagrams added.** The original lists them under Key Ideas but has no slides; Part 2 is new.
- **Include arrow direction corrected.** The original rule (“arrow with the use case that completes first”) puts the include arrow at the base. Standard UML points it to the included use case. The extend example was already correct.
- **Secondary actor.** Kept as defined in the original (administrative user). Many textbooks use the term for a supporting system instead.
- **External hardware example replaced.** “Amazon datastore as database” is a software service; replaced with an ID-card scanner.
- **System boundary wording.** “Class in which use cases are executed” reworded as “the system that carries out the use cases”.
- **Action verbs.** “Every action verb is a use case” taught as “verbs are candidate use cases”.
- **Examples.** Ride-sharing, kitchen and bank examples replaced by one course-registration example; the bank, hospital and process-order figures are kept in the exercise bank and appendix.
- **SAF exercise.** Exactly Tk 10,000 is not covered by the original wording (under / over).